LAPORAN TUGAS BESAR 1
IF3170 INTELIGENSI ARTIFISIAL



Disusun oleh:
Agatha Tatianingseto - 13524008
Reva Natania Sitohang - 13524098
Amanda Aurellia Salsabilla - 13524131

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2026

DAFTAR ISI

BAB I
PENDAHULUAN..... 3
1.1 Latar Belakang..... 3
1.2 Gambaran Umum Persoalan..... 3
BAB II
DASAR TEORI.....4
2.1 Local Search.....4
2.2 Hill Climbing.....4
2.2.1 Sideways Move Hill Climbing..... 5
2.2.2 Random Restart Hill Climbing..... 5
2.2.3 Stochastic Hill Climbing.....5
2.3 Simulated Annealing..... 6
2.4 Genetic Algorithm.....6
BAB III
DESKRIPSI PERSOALAN.....8
2.1 Gambaran Umum Persoalan..... 8
2.2 Gambaran Umum Persoalan..... 8
2.3 Representasi State..... 9
2.4 Initial State..... 9
2.5 Pembentukan Neighbor.....9
2.6 Objective Function.....10
2.7 Constraints..... 10
2.8 Output Sistem.....11
BAB IV
IMPLEMENTASI ALGORITMA..... 12
4.1 Hill Climbing.....12
4.1.1 Steepest Ascent..... 13
4.1.2 Sideways Move.....14
4.1.3 Random Restart..... 15
4.1.4 Stochastic Hill Climbing..... 16
4.2 Simulated Annealing..... 17
4.3 Genetic Algorithm.....20
4.3.1 Gambaran Umum..... 20
4.3.2 Seleksi Parent.....22
4.3.3 Mutasi..... 23
4.3.4 Crossover.....25

4.3.5 Fungsi search..... 26
4.3.6 Heuristik yang Digunakan..... 29
4.4 Bonus..... 30
4.4.1 Objective ETA-weighted..... 30
4.4.2 Multi-Truk..... 30
BAB V
EKSPERIMEN DAN HASIL..... 31
5.1 Hill Climbing..... 31
5.1.1 Steepest Ascent Hill Climbing..... 31
5.1.2 Sideways Move Hill Climbing..... 34
5.1.3 Random Restart Hill Climbing..... 37
5.1.4 Stochastic Hill Climbing..... 39
5.2 Simulated Annealing..... 41
5.3 Genetic Algorithm..... 47
5.3.1 Pengaturan Eksperimen..... 47
5.3.2 Hasil Populasi 50 dan Iterasi 100..... 48
5.3.3 Eksperimen A: Variasi Iterasi (populasi 50, medium, value)..... 54
5.3.4 Eksperimen B: Variasi Populasi (iterasi 100, medium, value)..... 55
BAB VI
ANALISIS..... 57
6.1 Kedekatan ke Lokal Optima..... 57
6.1.1 Hill Climbing..... 57
6.1.2 Simulated Annealing..... 61
6.1.3 Genetic Algorithm..... 62
6.2 Dinamika Proses Pencarian..... 64
6.2.1 Komposisi keputusan per iterasi..... 64
6.2.2 Probabilitas penerimaan e^(ΔE/T)..... 64
6.2.3 Frekuensi stuck..... 65
6.2.4 Dinamika Pencarian Genetic Algorithm..... 66
6.2.5 Dinamika Pencarian Steepest Ascent Hill Climbing..... 67
6.2.6 Dinamika Pencarian Sideways Move Hill Climbing..... 67
6.2.7 Dinamika Pencarian Random Restart Hill Climbing..... 68
6.2.8 Dinamika Pencarian Stochastic Hill Climbing..... 69
6.3 Pengaruh Iterasi dan Jumlah Populasi pada GA..... 69
6.4 Perbandingan Antar Algoritma..... 70
6.5 Pengaruh Objective ETA-weighted dan Multi-Truk..... 71
6.6 Keterbatasan..... 72

BAB VII
KESIMPULAN DAN SARAN..... 73
7.1 Kesimpulan..... 73
7.2 Saran..... 74
PEMBAGIAN TUGAS..... 76
REFERENSI..... 77
FORMAT DEKLARASI PENGGUNAAN AI..... 78

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam kegiatan logistik, keterbatasan ruang dan kapasitas angkut membuat proses penyusunan paket di dalam peti kemas menjadi persoalan yang tidak sederhana. Setiap paket dapat memiliki ukuran, berat, nilai, tingkat kerapuhan, dan waktu kedatangan yang berbeda. Selain menentukan paket mana yang layak dimuat, perlu dipertimbangkan pula posisi, orientasi, kemungkinan tumpang tindih, dukungan antar paket, serta batas kapasitas peti kemas. Semakin banyak paket yang tersedia, semakin besar pula jumlah kemungkinan konfigurasi yang harus dipertimbangkan.

Tugas besar ini memodelkan persoalan tersebut sebagai permasalahan optimasi menggunakan pendekatan local search. Tiga algoritma, yaitu Hill Climbing, Simulated Annealing, dan Genetic Algorithm, digunakan untuk mencari konfigurasi pengisian peti kemas dengan nilai total paket setinggi mungkin tanpa melanggar constraint yang berlaku. Selain objective utama berbasis value, sistem juga menyediakan objective alternatif yang mempertimbangkan ETA sehingga perilaku pencarian dapat dianalisis dari lebih dari satu sudut pandang.

1.2 Gambaran Umum Persoalan

Persoalan dimodelkan dengan dua entitas. *Package* memiliki id, dimensi (w, l, h), orientasi, value, weight, status *fragile*, dan ETA. Truck memiliki dimensi peti kemas dan kapasitas berat maksimum. Tujuannya memaksimalkan total value paket yang berada di dalam peti kemas, dengan tiga batasan:

- paket tidak boleh melayang atau menembus dinding peti kemas;
- paket *fragile* tidak boleh menjadi penyangga paket lain;
- total berat muatan tidak boleh melebihi kapasitas truk.

State awal dibangkitkan secara acak. Langkah perpindahan yang diizinkan adalah menukar dua paket, memindahkan paket ke koordinat lain, dan merotasi paket 90° terhadap sumbu x, y, atau z.

BAB II
DASAR TEORI

2.1 Local Search

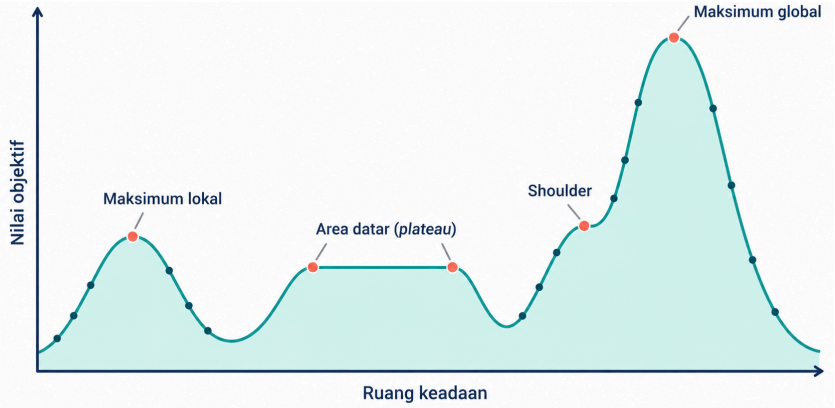
Local search dipakai ketika jalur urutan aksi (*path to goal* tidak relevan. Local search memakai formulasi complete state dimana setiap state sudah merupakan konfigurasi lengkap, hanya satu current state yang disimpan, dan aksinya berupa perpindahan ke neighbor. Tidak ada path cost maupun goal test, solusinya adalah final state dengan nilai terbaik.

Setiap state s memiliki nilai f(s) dari objective function (atau heuristic cost function), Fungsi successor menghasilkan semua state yang dapat dicapai satu langkah dari s. Neighbor yang dipilih dapat berupa highest-valued successor, dimana semua successor dievaluasi, atau random successor. Untuk persoalan maksimasi, tujuannya mencari s dengan f(s) tertinggi. Neighbor harus tetap berupa state yang valid. Ada dua pendekatan umum: menolak state yang melanggar constraint, atau memberi penalti pada objective. Implementasi tugas ini memakai pendekatan yang pertama.

Ruang state dapat digambarkan sebagai lanskap dengan state dan nilai objective. Titik tertinggi adalah global maximum. Selain itu ada local maximum, flat local maximum, dan shoulder. Karena hanya melihat neighbor, local search dapat terjebak di local optimum tergantung initial state.

2.2 Hill Climbing

Hill Climbing melakukan pencarian dari 1 state awal, lalu membangkitkan *neighbor*, yaitu calon solusi yang diperoleh dari satu perubahan pada *state* tersebut. Pada persoalan *truck* ini, perubahan dapat berupa memindahkan, menukar, atau memutar paket. Pada algoritma Hill Climbing biasa, atau biasa disebut *steepest ascent*, seluruh *neighbor* yang valid dinilai menggunakan fungsi objektif. Pemilihan *neighbor* adalah calon dengan skor tertinggi dan perpindahan hanya dilakukan apabila skornya lebih besar dari pada skor *state* saat ini. Langkah diulang sampai tidak ada *neighbor* yang lebih baik.



Akan tetapi pencarian dengan metode ini dapat berhenti pada *local maximum* atau daerah datar meskipun solusi yang lebih baik masih ada di bagian lain ruang pencairan. Sehingga dilakukan tiga pengembangan agar menghindari kondisi saat mencapai *local maximum* atau daerah datar, yaitu :

2.2.1 Sideways Move Hill Climbing

Sideways move membangkitkan dan menilai seluruh *neighbor* yang valid, kemudian memilih yang memiliki skor tertinggi. Apabila skor tertinggi lebih besar dari skor state saat ini, maka akan dipilih untuk berpindah pada state tersebut. Jika skor nya sama, perpindahan akan tetap dilakukan, hal ini dilakukan untuk menghindari daerah datar sehingga masih ada kemungkinan untuk menemukan jalur ke skor yang lebih tinggi. Jumlah untuk melakukan sideways move dibatasi, agar algoritma tidak terus berpindah di antara solusi dengan skor yang sama.

2.2.2 Random Restart Hill Climbing

Random Restart menjalankan Hill Climbing berulang kali dari state awal yang berbeda. Pada setiap pencarian, seluruh *neighbor* yang valid dinilai dan algoritma akan berpindah ke *neighbor* terbaik selama skornya meningkat. Apabila pencarian berhenti, algoritma akan membuat *state* awal *random* dan mulai mencari lagi. Dari seluruh pencarian tersebut, algoritma menyimpan state dengan skor tertinggi sebagai hasil akhir, dengan banyak nya *restart* dan langkah pada setiap pencarian yang dibatasi.

2.2.3 Stochastic Hill Climbing

Stochastic Hill Climbing membangkitkan 1 calon *neighbor* secara *random* pada setiap iterasi, selanjutnya diperiksa validitas dan skornya. Algoritma akan berpindah hanya jika calon *neighbor* valid dan memiliki skor lebih tinggi daripada *state* saat ini, kalau tidak maka akan menggunakan state saat ini. Proses diulang hingga batas iterasi yang ditentukan.

2.3 Simulated Annealing

Simulated Annealing menggabungkan hill-climbing yang efisien tapi masih mudah terjebak di local maksimum dengan *random walk* yang lengkap tetapi sangat tidak efisien. Idenya keluar dari local maximum dengan mengizinkan sebagian langkah buruh, lalu mengurangi frekuensinya secara bertahap. SA adalah versi stochastic hill-climbing yang memperbolehkan *downhill move* dengan temperatur sebagai parameter. Pada setiap iterasi, SA membangkitkan satu random successor dari state saat ini lalu menghitung selisih nilai:

$$\Delta E = f(s_{next}) - (s_{current})$$

- Jika ΔE > 0 (neighbor lebih baik), langkah selalu diterima, seperti stochastic hill-climbing.
- Jika ΔE < 0 (neighbor lebih buruk), langkah diterima dengan probabilitas berikut

$$P(accept) = e^{\Delta E/T}$$

Keputusan diambil dengan membandingkan probabilitas tersebut dengan bilangan acak pada rentang (0, 1): langkah diterima jika bilangan acak lebih kecil dari probabilitas. T menurun seiring waktu sesuai cooling schedule. Saat T tinggi, langkah buruk mudah diterima sehingga pencarian mendekati random walk. Saat T mendekati nol, langkah buruk hampir selalu ditolak sehingga SA berperilaku seperti stochastic hill-climbing, semakin besar penurunan nilai |Δ E|, semakin kecil peluang langkah tersebut diterima pada temperatur yang sama.

Skema pendinginan memetakan waktu ke temperatur. Skema yang umum adalah geometrik (eksponensial), dengan faktor pendinginan α antara 0 dan 1:

$$T_{t+1} = \alpha T_t \Rightarrow T_t = T_0 \alpha^t$$

Pencarian berhenti ketika T mencapai batas minimum atau batas iterasi tercapai. Jika T diturunkan cukup lambat, SA akan menemukan global optimum dengan probabilitas mendekati 1. Ini bergantung pada pendinginan yang sangat lambat, sehingga pada jumlah iterasi terbatas hasilnya tetap bergantung pada pemilihan parameter.

2.4 Genetic Algorithm

Genetic Algorithm (GA) adalah algoritma local search yang terinspirasi dari proses seleksi alam [1]. Berbeda dengan hill climbing dan simulated annealing yang hanya

menyimpan satu current state, GA menyimpan sekumpulan state sekaligus yang disebut populasi, dan setiap state di dalamnya disebut individu. Kualitas individu diukur dengan *fitness function*, yang pada tugas ini sama dengan *objective function* sehingga semakin besar nilainya semakin baik.

Pencarian berlangsung dalam beberapa generasi. Pada setiap generasi ada tiga proses utama. Pertama seleksi, yaitu memilih individu yang akan menjadi parent dengan peluang yang lebih besar bagi individu ber-fitness tinggi. Kedua *crossover*, yaitu menggabungkan dua parent menjadi satu child sehingga bagian yang baik dari kedua parent dapat diwariskan. Ketiga mutasi, yaitu mengubah child sedikit secara acak agar muncul variasi yang belum ada pada populasi sebelumnya. Kumpulan child membentuk populasi generasi berikutnya, lalu proses diulang sampai batas generasi tercapai.

Pada implementasi ini dipakai dua teknik tambahan. *Tournament selection* memilih parent dengan mengadu beberapa individu acak dan mengambil yang terbaik, dengan ukuran turnamen yang mengatur kuat lemahnya tekanan seleksi. *Elitism* menyalin individu terbaik langsung ke generasi berikutnya sehingga skor terbaik tidak pernah turun. Karena persoalan ini memiliki *constraint*, setiap individu harus tetap berupa state yang valid, sehingga seluruh operator dirancang dengan pendekatan menolak state yang melanggar constraint.

BAB III
DESKRIPSI PERSOALAN

2.1 Gambaran Umum Persoalan

Persoalan yang diselesaikan pada tugas besar ini adalah optimasi penempatan sejumlah paket ke dalam peti kemas sebuah truk. Setiap paket memiliki karakteristik yang berbeda, seperti ukuran, nilai, berat, sifat pecah-belah, dan Estimated Time of Arrival (ETA). Paket tidak wajib seluruhnya dimasukkan ke dalam peti kemas sehingga solusi perlu menentukan paket mana yang dimuat, posisi paket di dalam peti kemas, serta orientasinya. Tujuan utama optimasi adalah memperoleh konfigurasi pengisian dengan total value sebesar mungkin.

Permasalahan direpresentasikan sebagai ruang pencarian yang terdiri atas berbagai konfigurasi paket. Dari suatu konfigurasi, algoritma local search dapat berpindah menuju konfigurasi lain melalui operasi tertentu. Seluruh konfigurasi yang dihasilkan tetap harus memenuhi batasan fisik peti kemas dan karakteristik paket. Dengan demikian, solusi terbaik tidak hanya ditentukan oleh nilai objective yang tinggi, tetapi juga oleh validitas susunan paket tersebut.

2.2 Gambaran Umum Persoalan

1. Truck

   Truck merepresentasikan peti kemas yang menjadi ruang penempatan paket. Informasi yang disimpan meliputi:

   - `dimensions` , berupa width, length, dan height peti kemas;
   - `max_capacity` , yaitu batas maksimum total berat paket yang dapat diangkut.

   Sebagai contoh, peti kemas dengan dimensi 10 × 20 × 5 menyediakan ruang tiga dimensi dengan lebar 10, panjang 20, dan tinggi 5 satuan. Kapasitas berat diperiksa secara terpisah dari kapasitas ruang.

2. Package

   Setiap package memiliki atribut:

   - `id` , sebagai identitas paket;
   - `dimensions` , berupa width, length, dan height;
   - `orientation` , yang menunjukkan keterkaitan sisi paket terhadap sumbu x , y , dan z;

- `value` , yaitu nilai paket yang digunakan dalam perhitungan objective;
- `weight` , yaitu berat paket;
- `is_fragile` , yang menunjukkan apakah paket merupakan barang pecah-belah;
- `ETA` , yaitu estimasi waktu kedatangan paket dalam satuan hari.

Paket berbentuk kubus atau balok dan dapat memiliki ukuran yang berbeda-beda. Paket juga dapat dirotasi sehingga dimensi aktualnya terhadap masing-masing sumbu dapat berubah.

2.3 Representasi State

Satu state merepresentasikan satu konfigurasi lengkap dari seluruh paket yang tersedia. Pada implementasi yang dibuat, state menyimpan pemetaan setiap `pacakge_id` terhadap sebuah Placement.

Sebuah Placement menyimpan:

- `truck_id`;
- `position(x, y, z)`;
- `orientation` .

Apabila nilai placement suatu paket adalah None, paket tersebut dianggap berada di luar peti kemas.

2.4 Initial State

Proses pencarian dimulai dari initial state yang dibangkitkan secara acak. Pada implementasi sistem, initial state dibuat dengan mencoba menempatkan paket pada posisi dan orientasi tertentu secara acak. Hanya konfigurasi yang lolos proses validasi yang dipertahankan. Penggunaan seed juga disediakan agar pembangkitan state dan proses pencarian dapat direproduksi pada eksperimen yang sama

2.5 Pembentukan Neighbor

Neighbor merupakan state baru yang dicapai dari current state melalui satu perubahan. Terdapat tiga jenis operasi yang digunakan.

1. Swap
   Menukar placement dua paket. Pertukaran dapat terjadi antara dua paket di dalam peti kemas maupun antara paket di dalam dan di luar peti kemas.

2. Move

Memindahkan sebuah paket ke koordinat lain, memasukkan paket yang sebelumnya berada di luar, atau mengeluarkan paket dari peti kemas.

3. Rotate
   Merotasi paket sebesar 90° terhadap sumbu x, y, atau z, sehingga orientasi dan dimensi aktual paket dapat berubah.

Pada implementasi Simulated Annealing, neighbor dipilih secara acak menggunakan salah satu dari ketiga operasi tersebut. Candidate state hanya digunakan apabila memenuhi seluruh aturan validasi.

2.6 Objective Function

Objective utama pada tugas besar adalah memaksimalkan total value seluruh paket yang berhasil dimuat ke dalam peti kemas. Secara umum:

$$f(s) = \sum_{p \in Loaded(s)} value(p)$$

dengan *Loaded(s)* merupakan himpunan paket yang berada di dalam peti kemas pada state s. Semakin besar nilai *f(s)*, semakin baik state tersebut.

Selain objective utama, implementasi juga menyediakan objective alternatif berbasis ETA sebagai tambahan. Objective ini memberikan bobot tambahan kepada paket berdasarkan ETA sehingga paket dengan waktu kedatangan yang lebih mendesak (lebih kecil) dapat memperoleh prioritas yang lebih tinggi.

2.7 Constraints

Setiap state harus memenuhi sejumlah constraint agar dianggap valid.

1. Batas Peti Kemas
   Seluruh bagian paket harus berada di dalam peti kemas. Paket tidak diperbolehkan berada sebagian di dalam dan sebagian di luar peti kemas.

2. Kapasitas Berat
   Jumlah berat seluruh paket yang dimuat tidak boleh melebihi `max_capacity` peti kemas.

$$\sum_{p \in Loaded(s)} weight(p) \leq max_capacity$$

3. Support

Paket tidak diperbolehkan melayang. Sebuah paket memiliki support apabila berada langsung pada lantai peti kemas atau terdapat bagian atas paket lain tepat di bawah setidaknya sebagian alasnya.

4. Fragile Package
   Paket dengan isi `is_fragile = true` tidak diperbolehkan menjadi support untuk paket lain.

5. Overlap
   Pada implementasi sistem, dua paket juda tidak diperbolehkan menempati ruang tiga dimensi yang sama. Pemeriksaan overlap dilakukan oleh modul validasi sebelum sebuah state diterima sebagai candidate yang lain.

2.8 Output Sistem

Hasil pencarian berupa final state terbaik yang ditemukan oleh algoritma. State tersebut menunjukkan:

- paket yang berada di dalam peti kemas;
- paket yang berada di luar peti kemas;
- posisi setiap paket;
- orientation setiap paket;
- orientation setiap paket;
- nilai objective akhir.

Program juga menyediakan visualisasi tiga dimensi dari initial state dan final state serta grafik perkembangan objective selama proses pencarian. Informasi eksperimen lain, seperti jumlah iterasi dan waktu pencarian, disimpan untuk kebutuhan analisis antara algoritma. Hal ini sesuai dengan ketentuan bahwa program harus mampu memvisualisasikan initial state, final state, dan hasil eksperimen

BAB IV
IMPLEMENTASI ALGORITMA

4.1 Hill Climbing

HIll Climbing menerima state awal yang dibangkutkan oleh run(), dimana fungsi search() memilih pendekatan yang dijalankan berdasarkan parameter variant, yaitu steepest, sideways, random_restart, atau stochastic. Pemilihan acak menggunakan seed sehingga percobaan dengan input dan parameter yang sama dapat diulang.

Fungsi best_neighbors() membangkitkan *neighbor* melalui operasi pada neighbor.py, kemudian menghitung skor setiap *neighbor*, dan menyimpan semua state yang memiliki skor tertinggi. Apabila beberapa *neighbor* memiliki skor terbaik yang sama, salah satu dipilih secara acak oleh algoritma.

Algoritma best_neighbors():

```python
def best_neighbors(problem: Problem, current: State, objective: ObjectiveConfig):
    best_score = float("-inf")
    best_state = []
    evaluated = 0
    for candidate in neighbors(problem, current):
        candidate_score = score(problem, candidate, objective)
        evaluated += 1
        if candidate_score > best_score:
            best_score = candidate_score
            best_state = [candidate]
        elif candidate_score == best_score:
            best_state.append(candidate)
    return best_state, best_score, evaluated
```

Parameter yang digunakan dalam implementasi adalah:

Parameter	Default	Keterangan
variant	steepest	Pendekatan yang dipilih: steepest, sideways, random_restart, atau stochastic.

max_iterations	1000	Batas iterasi untuk steepest, sideways, dan stochastic.
max_sideways_moves	100	Batas sideways move berturut-turut pada pendekatan sideways.
max_restarts	10	Banyaknya pencarian ulang setelah pencarian awal.
max_iterations_per_restart	1000	Batas langkah peningkatan pada setiap pencarian dalam random_restart.
seed	None	Seed untuk pembangkitan state awal dan pemilihan acak.
objective	value	Mode penilaian; eta_weighted menggunakan parameter eta_bonus.
eta_bonus	0.0	Besar bonus ETA pada mode eta_weighted.

4.1.1 Steepest Ascent

Pada setiap iterasi, fungsi best_neighbors() akan membangkitkan semua neighbor valid, menghitung skornya, dan menyimpan semua neighbor dengan skor tertinggi. Apabila terdapat berapa kandidat dengan skor tertinggi yang sama, salah satunya dipilih secara acak. Fungsi juga menghitung banyaknya neighbor yang dievaluasi.

Algoritma :

```python
for _ in range(max_iterations):
    best_states, best_score, count = best_neighbors(problem, current, objective)
    evaluated += count
    if not best_states or best_score <= current_score:break

    current = rng.choice(best_states)
    current_score = best_score
    history.append(current_score)
```

Alur pencariannya:

1. State awal disalin menjadi current, lalu skornya dihitung dan dicatat sebagai skor pertama pada history.
2. Pada setiap iterasi, seluruh neighbor valid dievaluasi oleh best_neighbors().
3. Jika tidak ada neighbor atau skor terbaik tidak lebih tinggi dari skor current, pencarian berhenti.

4. Jika skor meningkat, salah satu neighbor terbaik dipilih, current diperbarui, dan skor barunya dicatat pada history.
5. Pencarian juga berhenti setelah max_iterations langkah. Hasilnya adalah current beserta riwayat skor dan jumlah neighbor yang dievaluasi.

4.1.2 Sideways Move

Pendekatan ini menggunakan pembangkitan dan pemilihan *neighbor* terbaik yang sama dengan *Steepest Ascent*, tetapi memperbolehkan perpindahan ketika skor *neighbor* sama dengan skor saat ini.

Algoritma :

```python
for _ in range(max_iterations):
    best_states, best_score, count = best_neighbors(problem, current, objective)
    evaluated += count
    if not best_states or best_score < current_score:
        break
    if best_score == current_score and consecutive_sideways >= max_sideways_moves:
        break
    current = rng.choice(best_states)
    if best_score == current_score:
        sideways_moves += 1
        consecutive_sideways += 1
    else:
        consecutive_sideways = 0
    current_score = best_score
    history.append(current_score)
```

Alur pencariannya:

1. Algoritma menyiapkan current, skor awal, dan history, serta penghitung jumlah sideways move.
2. Pada setiap iterasi, seluruh neighbor valid dievaluasi oleh best_neighbors().
3. Pencarian berhenti jika tidak ada neighbor atau skor neighbor terbaik lebih rendah daripada skor saat ini.

4. Jika skornya sama, perpindahan hanya dilakukan selama jumlah sideways move berturut-turut belum mencapai max_sideways_moves. Jika skornya meningkat, penghitung berturut-turut direset.
5. Skor setiap perpindahan dicatat ke history. Proses berhenti sesuai kondisi di atas atau saat max_iterations tercapai.

4.1.3 Random Restart

Random Restart menjalankan pencarian Hill Climbing beberapa kali dari state awal yang berbeda. Variable best menyimpan solusi terbaik dari sleuruh pencarian sedangkan current menyimpan state yang sedang di telusuri.

Algoritma :

```python
for run_index in range(max_restarts + 1):
    steps = 0
    current_score = score(problem, current, objective)
    if current_score > best_score:
        best = current.copy()
        best_score = current_score
        history[-1] = best_score
    for _ in range(max_iterations_per_restart):
        best_states, next_score, count = best_neighbors(problem, current,
objective)
        evaluated += count
        if not best_states or next_score <= current_score:
            break

        current = rng.choice(best_states)
        current_score = next_score
        steps += 1
        if current_score > best_score:
            best, best_score = current.copy(), current_score
        history.append(best_score)

    iterations_per_restart.append(steps)
    if best_score >= upper_bound or run_index == max_restarts:
```

```
            break
        current = initial_state(problem, seed=rng.randrange(2**63))
        restarts += 1
        history.append(best_score)
```

Alur pencariannya:

1. Algoritma menyimpan state awal sebagai best dan current, lalu menghitung batas skor upper_bound dari jumlah nilai seluruh paket sesuai mode fungsi objektif.
2. Untuk setiap pencarian, skor state awalnya dibandingkan dengan best_score. Sehingga hasil dari state awal baru tetap diperhitungkan meskipun tidak ada langkah lanjutan yang meningkatkan skor.
3. Algoritma mengevaluasi seluruh neighbor valid dan berpindah ke salah satu neighbor terbaik hanya jika skornya meningkat. Setiap peningkatan yang melampaui best_score memperbarui best.
4. Ketika pencarian berhenti, algoritma mencatat jumlah langkah yang dilakukan. Jika batas skor tercapai atau jatah restart habis, proses berakhir. Jika belum, initial_state() membangkitkan state acak baru untuk pencarian berikutnya.
5. Hasil akhirnya adalah best, yaitu solusi terbaik dari seluruh pencarian. history berisi skor terbaik sejauh ini, termasuk ketika sebuah restart dilakukan.

4.1.4 Stochastic Hill Climbing

Pendekatan stokastik menggunakan random_neighbor() untuk mencoba satu operasi acak, yaitu move, swap, atau rotate. Fungsi tersebut mencoba maksimal 64 kali untuk mendapatkan neighbor valid dan mengembalikan None jika seluruh percobaan gagal.

Algoritma :

```
Python
for _ in range(max_iterations):
    candidate = random_neighbor(problem, current, rng)
    if candidate is not None:
        candidate_score = score(problem, candidate, objective)
        if candidate_score > current_score:
            current, current_score = candidate, candidate_score
```

```
            accepted += 1
    history.append(current_score)
```

Alur pencariannya:

1. Algoritma menyalin state awal menjadi current dan mencatat skor awalnya.
2. Pada setiap iterasi, random_neighbor() mencoba membangkitkan satu neighbor valid.
3. Jika neighbor ditemukan, skornya dihitung. Perpindahan diterima hanya ketika skor neighbor lebih besar daripada skor saat ini; jumlah perpindahan yang diterima dicatat pada accepted.
4. Jika tidak ada neighbor valid atau skornya tidak meningkat, current tetap. Skor saat ini tetap ditambahkan ke history, sehingga setiap iterasi memiliki catatan.
5. Setelah max_iterations percobaan, algoritma mengembalikan state terakhir, riwayat skor, dan jumlah perpindahan yang diterima.

4.2 Simulated Annealing

Urutan awal paket diacak dengan initial_state. Setiap paket dilewati dengan peluang 0,25. Untuk paket lainnya, kandidat posisi dan orientasi diacak, lalu dipakai kandidat pertama yang valid. Satu operasi dipilih random_neighbor secara seragam dari tiga operasi swap, move, atau rotate. Kandidat yang tidak valid dibuang, dan pemilihan operasi diulang hingga max_neighbor_attempts kali. Jika semua percobaan gagal, fungsi mengembalikan None. SA memakai satu random successor per iterasi, sehingga seluruh neighbor tidak perlu dienumerasi seperti pada fungsi neighbors() untuk hill-climbing.

Setiap neighbor yang dikembalikan sudah lolos is_valid, yaitu paket berada dalam batas truk, tidak saling tumpang tindih, total berat tidak melebihi kapasitas, setiap paket tidak melayang, dan paket fragile tidak menjadi penyangga. SA hanya bergerak di antara state yang feasible, sehingga objective tidak memerlukan penalti. SA memaksimalkan f(s), dengan L(s) adalah himpunan paket yang berada di dalam truk pada state s:

$$f_{value}(s) = \sum_{i \in L(s)} v_i \left(1 + \frac{\beta}{eta_i + 1}\right)$$

Di sini v_i adalah value paket dan β adalah eta_bonus. Mode eta_weighted memberi bobot lebih pada paket dengan ETA kecil.

```python
# disingkat dari search() pada simulated_annealing.py
delta = candidate_score - current_score
if delta >= 0 or rng.random() < math.exp(delta / temperature):
    current_state, current_score = candidate_state, candidate_score

if current_score > best_score:
    best_state, best_score = current_state.copy(), current_score

temperature *= cooling_rate
```

Alur algoritmanya adalah:

1. run() membangkitkan state awal acak. SA menyalinnya menjadi current_state dan best_state, menghitung skornya, lalu menetapkan temperatur awal T0.
2. Pada setiap iterasi, random_neighbor menghasilkan satu neighbor valid. Jika tidak ditemukan dalam max_neighbor_attempts percobaan, iterasi dicatat sebagai neighbor_failures dan state tidak berubah.
3. Skor neighbor dihitung dengan score, lalu ΔE diperoleh dari skor neighbor dikurangi skor state saat ini.
4. Neighbor dengan ΔE > 0 selalu diterima. Neighbor dengan ΔE = 0 juga diterima agar pencarian dapat melintasi daerah datar (flat atau shoulder). Neighbor dengan ΔE < 0 diterima jika bilangan acak pada (0, 1) lebih kecil dari e^(ΔE/T) dan jika tidak, neighbor ditolak. Keempat kemungkinan ini dihitung pada accepted_better, accepted_equal, accepted_worse, dan rejected.
5. current_state dan best_state dipisah karena SA dapat menerima langkah buruk, sehingga state terakhir belum tentu yang terbaik. Jika skor current melampaui skor best, best diperbarui dan penghitung iterasi tanpa perbaikan direset. Jika penghitung mencapai stuck_threshold, stuck_frequency bertambah satu untuk episode tersebut.
6. Temperatur dikalikan cooling_rate (pendinginan geometrik), jumlah iterasi bertambah, lalu skor current, skor best, dan temperatur dicatat ke riwayat.
7. Loop berhenti saat max_iterations tercapai atau temperatur ≤ min_temperature. Yang dikembalikan adalah best_state.

Parameter yang digunakan pada implementasi adalah:

Parameter	Default	Keterangan
`initial_temperature`	1000.0	Temperatur awal T0, harus lebih dari 0
`cooling_rate`	0.995	Faktor pendinginan α, 0 < α < 1
`min_temperature`	0.01	Pencarian berhenti jika T ≤ nilai ini, harus lebih dari 0
`max_iterations`	1000	Batas jumlah iterasi
`max_neighbor_attempts`	100	Percobaan maksimum membangkitkan satu neighbor valid per iterasi
`stuck_threshold`	50	Jumlah iterasi berturut-turut tanpa perbaikan best agar dihitung sebagai stuck
`seed`	`None`	Seed generator acak untuk reproduksibilitas (juga dipakai pada state awal)
`objective`	`value`	Mode objective, dengan eta_bonus bila eta_weighted

Parameter yang tidak valid memicu ValueError. Dengan nilai default, pencarian berhenti karena max_iterations: setelah 1000 iterasi T masih sekitar 6,65, dan T baru mencapai min_temperature setelah sekitar 2.297 iterasi. Nilai T0 juga perlu disesuaikan dengan skala ΔE; pada mode eta_weighted skor dapat mencapai puluhan ribu, sehingga T0 yang sama menjadi relatif kecil.

Metrics dan keluaran yang dipakai adalah sebagai berikut:

- score_history: riwayat best-so-far (panjang iterations + 1), sehingga nilai terakhirnya sama dengan skor final_state. current_score_history menyimpan skor state saat ini yang dapat turun, dan temperature_history menyimpan temperatur tiap iterasi. acceptance_probability_history menyimpan e^(ΔE/T) tiap iterasi, bernilai null jika langkahnya tidak lebih buruk (ΔE ≥ 0) atau tidak ada neighbor valid.

- accepted_better, accepted_equal, accepted_worse, rejected, dan neighbor_failed menghitung hasil keputusan pada setiap iterasi. acceptance_rate adalah jumlah langkah diterima dibagi jumlah iterasi.
- stuck_frequency: banyaknya episode stuck_threshold iterasi berturut-turut tanpa best baru. Satu episode hanya dihitung sekali dan direset ketika best baru ditemukan. Definisi ini ditentukan kelompok karena yang diukur adalah stagnasi skor best, bukan kondisi local optimum. Penghitung bertambah pada setiap iterasi tanpa perbaikan best, termasuk saat langkah buruk diterima atau ditolak dan saat tidak ada neighbor valid. Ambang 100 iterasi (5% dari 2.000 iterasi) adalah pilihan kelompok dan dipakai sama pada semua run. Definisi ini mengikuti cara local search menilai kemajuan, yaitu perbaikan nilai objective. Hill-climbing berhenti ketika tidak ada neighbor yang lebih baik, dan varian sideways move membatasi jumlah langkah berturut-turut tanpa perbaikan. Stuck di sini adalah bentuk yang sama pada SA: jumlah iterasi berturut-turut tanpa perbaikan, yang lazim dipakai untuk mendeteksi stagnasi pada algoritma iteratif. Pengukuran memakai skor best, bukan skor current. SA sengaja menerima langkah turun, sehingga skor current naik-turun dan penurunannya tidak sama dengan stagnasi. Skor best tidak pernah turun, sehingga “tidak ada kemajuan” terdefinisi jelas dan sejalan dengan hasil akhir yang dilaporkan, yaitu state best. Satu episode dihitung sekali agar stagnasi panjang tidak menggelembungkan frekuensi. Yang dibandingkan adalah banyaknya periode stagnasi, bukan lamanya.
- run() menambahkan state dan skor awal, state dan skor akhir, serta durasi pencarian.

4.3 Genetic Algorithm

4.3.1 Gambaran Umum

Genetic Algorithm diimplementasikan pada berkas genetic_algorithm.py dan didaftarkan ke sistem dengan register("genetic_algorithm", search). Satu individu adalah satu State, yaitu pemetaan setiap package_id ke sebuah Placement atau None. Fitness individu dihitung dengan fungsi score yang sama dengan algoritma lain, sehingga objective value maupun eta_weighted bisa dipakai tanpa perubahan.

Setiap individu selalu berupa state yang valid. State awal dibuat oleh initial_state yang hanya mempertahankan konfigurasi valid, mutasi memakai move, rotate, dan swap yang mengembalikan None bila hasilnya tidak valid, dan

crossover hanya menyisipkan paket bila is_valid bernilai benar. Populasi awal terdiri dari state initial yang diberikan run() sebagai individu pertama, ditambah population_size - 1 individu dari initial_state dengan seed yang diambil dari generator acak ber-seed. Dengan begitu, run dengan seed yang sama menghasilkan hasil yang sama.

Alur satu generasi:

1. Individu terbaik disalin ke populasi baru (*elitisme*), sehingga skor terbaik tidak pernah turun.
2. Selama populasi baru belum penuh, parent pertama dipilih dengan *tournament selection*.
3. Dengan peluang crossover_rate, parent kedua dipilih dengan turnamen lain, lalu crossover menghasilkan child. Jika tidak, child adalah salinan parent pertama.
4. Dengan peluang mutation_rate, child dimutasi dengan satu operasi acak.
5. Child dimasukkan ke populasi baru. Setelah penuh, skor seluruh individu dihitung ulang dan skor maksimum serta rata-rata populasi dicatat.
6. Langkah 1 sampai 5 diulang sebanyak max_iterations, lalu individu terbaik dikembalikan sebagai *final state*.

Parameter yang digunakan:

Parameter	Default	Keterangan
population_size	50	Jumlah individu dalam populasi, minimal 1
max_iterations	100	Jumlah generasi, tidak boleh negatif
tournament_size	2	Jumlah peserta tiap turnamen, minimal 1
mutation_rate	0,7	Peluang sebuah child dimutasi, antara 0 dan 1
crossover_rate	0,8	Peluang dua parent digabung, antara 0 dan 1
seed	None	Seed untuk reproduksibilitas, juga dipakai membuat populasi awal

objective	value	Mode penilaian; eta_weighted memakai eta_bonus

Nilai parameter yang tidak valid memunculkan ValueError. Selain riwayat skor, fungsi mencatat *metrics* berupa mutation_count, mutation_success, crossover_count, dan crossover_success. Kata success di sini berarti hasil operasi berbeda dari state asalnya (untuk *crossover*, berbeda dari kedua parent), bukan berarti skornya naik.

4.3.2 Seleksi Parent

Seleksi dilakukan oleh select_parent dengan *tournament selection*. Fungsi mengambil tournament_size indeks individu secara acak tanpa pengembalian, membandingkan skor tiap peserta, lalu mengembalikan individu dengan skor tertinggi. Jika tournament_size lebih besar dari jumlah individu, nilainya dibatasi sama dengan jumlah individu agar sample tidak error. Bila ada skor yang sama, peserta yang terambil lebih dulu yang menang, karena perbandingannya memakai tanda lebih besar saja. Ukuran turnamen kecil membuat individu yang kurang baik masih punya peluang menjadi parent sehingga variasi populasi lebih terjaga, sedangkan ukuran besar membuat individu terbaik hampir selalu terpilih.

Kode program:

```
# tournament selection
# ambil beberapa individu acak, yang skornya paling tinggi jadi parent
def select_parent(population, scores, rng, tournament_size):
    population_size = len(population)
    tournament_size = min(tournament_size, population_size)
              participant_indexes    =    rng.sample(range(population_size),
tournament_size)

    winner_index = participant_indexes[0]
    winner_score = scores[winner_index]
    for participant_index in participant_indexes:
        participant_score = scores[participant_index]
        if participant_score > winner_score:
            winner_index = participant_index
            winner_score = participant_score
```

```
    return population[winner_index]
```

4.3.3 Mutasi

Fungsi random_move dipakai oleh operasi move. Fungsi memilih satu truk acak, lalu mengambil orientasi paket saat ini (atau orientasi awal bila paket berada di luar). Jika dimensi paket pada orientasi tersebut melebihi dimensi truk di salah satu sisi, fungsi mengembalikan None. Jika muat, koordinat dipilih acak pada tiap sumbu dari 0 sampai selisih dimensi truk dan dimensi paket, sehingga paket tidak menembus dinding. Contohnya, bila lebar truk 10 dan lebar paket 4, koordinat x yang mungkin adalah 0 sampai 6. Perpindahannya lalu diserahkan ke move, yang mengembalikan None bila state hasilnya tidak valid, misalnya paket tumpang tindih, melayang, atau berat melebihi kapasitas.

Fungsi mutate mengubah state dengan satu operasi acak. Setiap percobaan memilih satu dari empat operasi dengan peluang sama (move, rotate, swap, remove) dan satu paket acak. Operasi remove mengeluarkan paket dari truk. Kandidat bernilai None berarti operasinya gagal, sehingga fungsi mencoba lagi sampai max_attempts (10) kali dan mengembalikan kandidat valid yang pertama. Bila semua percobaan gagal, yang dikembalikan adalah salinan state tanpa perubahan, jadi pemanggil selalu menerima state yang valid.

Kode program:

```
# pindahin 1 paket ke posisi acak di truk acak
def random_move(problem, state, package, rng):
    truck = rng.choice(problem.trucks)

    current = state.placements[package.id]
    if current:
        orientation = current.orientation
    else:
        orientation = Orientation()

    size = oriented_dimensions(package, orientation)
    dims = truck.dimensions

    if size.w > dims.w or size.l > dims.l or size.h > dims.h:
        return None
```

```
    position = Position(
        rng.randrange(dims.w - size.w + 1),
        rng.randrange(dims.l - size.l + 1),
        rng.randrange(dims.h - size.h + 1),
    )

    return move(problem, state, package.id, truck.id, position)

# ubah state dengan 1 operasi acak, return state baru
def mutate(problem, state, rng, max_attempts=10):
    packages = problem.packages
    if not packages:
        return state.copy()

    for i in range(max_attempts):
        operation = rng.choice(("move", "rotate", "swap", "remove"))
        package = rng.choice(packages)

        if operation == "move":
            candidate = random_move(problem, state, package, rng)
        elif operation == "rotate":
            axis = rng.choice(("x", "y", "z"))
            candidate = rotate(problem, state, package.id, axis)
        elif operation == "swap":
            other = rng.choice(packages)
            candidate = swap(problem, state, package.id, other.id)
        else:
            candidate = move(problem, state, package.id, None)

        if candidate is not None:
            return candidate

    return state.copy()
```

4.3.4 Crossover

Crossover dilakukan per paket. Child dimulai dari state kosong (semua paket None), lalu diisi dengan placement yang diwarisi dari kedua parent selama hasilnya tetap valid.

Alur crossover:

1. Child dibuat kosong, semua paket berada di luar.
2. Untuk setiap paket, diundi dengan peluang 0,5 parent mana yang diutamakan. Daftar opsi placement disusun dari kedua parent sesuai urutan itu, dan hanya placement yang tidak None yang dimasukkan. Paket yang tidak ditempatkan oleh kedua parent tidak masuk daftar kandidat dan tetap di luar.
3. Daftar kandidat diacak, lalu diurutkan berdasarkan nilai z dari opsi pertama (fungsi first_option_z).
4. Kandidat diproses berurutan. Setiap opsi dipasang ke child, lalu dicek dengan is_valid. Jika valid, lanjut ke paket berikutnya. Jika tidak, placement dikembalikan ke None dan opsi berikutnya dicoba.
5. Paket yang semua opsinya ditolak tetap berada di luar.

Pengurutan dari z terkecil dimaksudkan agar paket yang berada di bawah dipasang lebih dulu, sehingga paket di atasnya memiliki support saat divalidasi. Daftar diacak terlebih dahulu supaya paket dengan z yang sama tidak selalu diproses dengan urutan yang sama. Placement diambil utuh dari parent termasuk truck_id, sehingga crossover juga bekerja pada kasus multi-truk. Karena paket yang bentrok dibuang, child cenderung kehilangan sebagian value dibanding parent-nya.

Kode program:

```
# ambil z dari pilihan pertama dari sebuah paket
def first_option_z(candidate):
    package_id, options = candidate
    first_placement = options[0]
    return first_placement.position.z

# gabungin 2 parent jadi 1 child
def crossover(problem, parent_a, parent_b, rng):
    empty_placements = {}
    for package in problem.packages:
        empty_placements[package.id] = None
    child = State(empty_placements)
```

```
    # tiap paket pilih parent yang dipakai duluan (acak), parent satunya
jadi cadangan
    candidates = []
    for package in problem.packages:
        if rng.random() < 0.5:
            sources = (parent_a, parent_b)
        else:
            sources = (parent_b, parent_a)

        options = []
        for source in sources:
            placement = source.placements[package.id]
            if placement is not None:
                options.append(placement)

        if options:
            candidates.append((package.id, options))

    # acak biar adil, lalu urutkan dari z kecil
    rng.shuffle(candidates)
    candidates.sort(key=first_option_z)

    for package_id, options in candidates:
        for placement in options:
            child.placements[package_id] = placement
            if is_valid(problem, child):
                break
            child.placements[package_id] = None

    return child
```

4.3.5 Fungsi search

Fungsi search menjalankan alur pada subbab 4.3.1. Skor seluruh individu dihitung oleh score_all_individuals, sehingga indeks skor selalu sama dengan indeks populasi. Riwayat skor terbaik dan rata-rata populasi dicatat satu kali untuk populasi awal dan satu kali setelah tiap generasi, jadi panjangnya max_iterations + 1 sesuai kontrak run(). Karena ada elitisme, skor terbaik pada riwayat tidak pernah turun.

Kode program:

```
# fungsi scoring
def score_all_individuals(problem, population, objective):
    scores = []
    for individual in population:
        scores.append(score(problem, individual, objective))
    return scores

def search(problem: Problem, initial: State, *, seed: int | None = None,
objective: ObjectiveConfig = ObjectiveConfig(), population_size: int = 50,
                  max_iterations: int = 100, tournament_size: int = 2,
mutation_rate: float = 0.7, crossover_rate: float = 0.8, **params) ->
SearchResult:
    if population_size < 1:
        raise ValueError("population_size harus minimal 1")
    if max_iterations < 0:
        raise ValueError("max_iterations tidak boleh negatif")
    if tournament_size < 1:
        raise ValueError("tournament_size harus minimal 1")
    if not 0 <= mutation_rate <= 1:
        raise ValueError("mutation_rate harus di antara 0 dan 1")
    if not 0 <= crossover_rate <= 1:
        raise ValueError("crossover_rate harus di antara 0 dan 1")

    rng = Random(seed)

    # populasi awal, seed acak
    population = [initial.copy()]
    for i in range(population_size - 1):
        individual_seed = rng.randrange(1, 2**31)
        population.append(initial_state(problem, individual_seed))

    scores = score_all_individuals(problem, population, objective)

    best_score = max(scores)
    best_index = scores.index(best_score)
    best_individual = population[best_index]
```

```
    best_score_history = [best_score]
    avg_score_history = [sum(scores) / len(scores)]

    crossover_count = 0
    crossover_success = 0
    mutation_count = 0
    mutation_success = 0

    for i in range(max_iterations):
        # elitisme: individu terbaik selalu lolos ke generasi berikutnya
        new_population = [best_individual.copy()]

        # sisanya diisi hasil seleksi
        while len(new_population) < population_size:
                    parent_a = select_parent(population, scores, rng,
tournament_size)

            if rng.random() < crossover_rate:
                        parent_b = select_parent(population, scores, rng,
tournament_size)
                crossover_count += 1
                child = crossover(problem, parent_a, parent_b, rng)
                if child != parent_a and child != parent_b:
                    crossover_success += 1
            else:
                child = parent_a.copy()

            if rng.random() < mutation_rate:
                mutation_count += 1
                mutated = mutate(problem, child, rng)
                if mutated != child:
                    mutation_success += 1
                child = mutated

            new_population.append(child)

        population = new_population
        scores = score_all_individuals(problem, population, objective)
```

```
        # karena ada elitisme, skor terbaik ga mungkin turun
        best_score = max(scores)
        best_index = scores.index(best_score)
        best_individual = population[best_index]

        best_score_history.append(best_score)
        avg_score_history.append(sum(scores) / len(scores))

       metrics = {"population_size": population_size, "max_iterations":
max_iterations, "tournament_size": tournament_size,
                        "mutation_rate": mutation_rate, "mutation_count":
mutation_count, "mutation_success": mutation_success,
                      "crossover_rate": crossover_rate, "crossover_count":
crossover_count, "crossover_success": crossover_success}

                      return     SearchResult(final_state=best_individual,
score_history=best_score_history,               iterations=max_iterations,
metrics=metrics, average_history=avg_score_history)
```

4.3.6 Heuristik yang Digunakan

Heuristik yang dipakai pada GA ini adalah *crossover* per paket yang divalidasi dan diurutkan dari z terkecil, mutasi dengan empat operasi yang peluangnya sama, serta pemilihan nilai *default* tournament_size 2 dan mutation_rate 0,7 berdasarkan percobaan pengembangan. Seluruhnya masih termasuk lingkup *local search*.

Nilai default tournament_size 2 dan mutation_rate 0,7 dipilih dari percobaan pendahuluan pada testcase medium dengan objective value, 10 seed, dan 100 iterasi. Pada populasi 60, kombinasi tournament 2 dan mutasi 0,7 menghasilkan rata-rata skor akhir 672 dengan 8 dari 10 seed mencapai 680, sedangkan kombinasi awal (tournament 3, mutasi 0,3) menghasilkan rata-rata 661 dengan 4 dari 10 seed mencapai 680. Percobaan ini dilakukan dengan skrip sementara sehingga angkanya bersifat indikatif.

4.4 Bonus

4.4.1 Objective ETA-weighted

Objective alternatif eta_weighted memberi nilai tiap paket yang termuat sebesar value dikalikan (1 + eta_bonus / (ETA + 1)), sehingga paket dengan ETA kecil

mendapat pengali lebih besar. *Objective* ini tidak menambah variabel baru, hanya parameter bonus. Eksperimen SA dan GA memakai eta_bonus = 300, sedangkan eksperimen HC memakai eta_bonus = 100 (dijelaskan pada subbab 6.1.1). Skala skornya jauh berbeda dari mode value (puluhan ribu dibanding ratusan), sehingga skor kedua mode tidak dibandingkan langsung. Perbandingan antar mode memakai value murni paket yang termuat dan rata-rata ETA paket yang termuat.

4.4.2 Multi-Truk

Parser menerima daftar truk, dan setiap Placement menyimpan truck_id. Validasi (batas truk, tumpang tindih, support, dan kapasitas berat) dihitung per truk, sedangkan *objective* menjumlahkan value seluruh paket yang termuat di semua truk. Pada GA, operasi move memilih truk tujuan secara acak sehingga paket dapat berpindah antar truk, dan crossover mewariskan placement lengkap termasuk truk-nya. Jumlah truk tidak dibatasi di kode, dan testcase yang dipakai memiliki 1, 2, dan 3 truk.

BAB V
EKSPERIMEN DAN HASIL

Eksperimen dilakukan pada 3 testcase dengan masing-masing testcase akan diuji menggunakan 3 seeds yang berbeda oleh masing-masing algoritma. Test yang digunakan serta deskripsi mengenai test case adalah sebagai berikut :

Name	Nama Test Case	Deskripsi
TC1	`small-1truck.json`	Terdiri atas 1 truk berdimensi 4 × 4 × 3 dengan kapasitas berat 30, serta 10 paket, termasuk 2 paket fragile. Menguji pemilihan paket pada kapasitas terbatas; salah satu paket memiliki berat yang menyamai seluruh kapasitas truk.
TC2	`medium-2trucks.json`	Terdiri atas 2 truk dengan dimensi 4 × 4 × 3 dan 4 × 3 × 2, berkapasitas berat 40 dan 20, serta 14 paket, termasuk 4 paket fragile. Menguji pembagian paket ant truk dengan ukuran dan kapasitas berbeda; beberapa paket hanya dapat dimuat di truk pertama karena beratnya melebihi kapasitas truk kedua.
TC3	`large-3trucks.json`	Terdiri atas 3 truk dengan dimensi 5 × 4 × 3, 4 × 4 × 3, dan 4 × 3 × 2, berkapasitas berat 45, 35, dan 20, serta 30 paket, termasuk 4 paket fragile. Menguji pencarian pada skala lebih besar dengan pilihan distribusi paket yang lebih banyak. Paket terberat berbobot 43, sehingga hanya dapat dimuat di truk pertama.

Selanjutnya diberikan hasil percobaan pada ketiga testcase dengan objective Value dan ETA Weighted. Visualisasi tiga dimensi hanya ditampilkan untuk testcase small dengan objective Value sebagai sampel. Berikut adalah hasil dari eksperimen yang dilakukan:

5.1 Hill Climbing

5.1.1 Steepest Ascent Hill Climbing

Objective :	**Value**			**ETA Weighted**		
Seeds :	42	43	44	42	43	44

TC1	Initial State	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)
	Final State	T1: P02(2,0,0;yzx) P06(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P05(3,1,1;xyz) P06(0,2,1;xyz) P09(0,1,0;zxy)	T1: P02(2,0,0;yzx) P06(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P04(1,1,0;yxz) P05(0,3,0;xyz) P06(0,0,0;xyz) P08(2,1,1;xyz) P09(0,1,0;zxy)
	Objective Awal	191	344	195	7.431	17.034	4.480,714
	Objective Akhir	228	344	266	9.828	17.034	17.034
	Durasi (s)	0,047	0,024	0,064	0,028	0,015	0,047
	Iterasi	1	0	2	1	0	4
TC2	Initial State	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)
	Final State	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P06(0,2,0;yxz) P07(0,1,0;xzy) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P05(0,1,2;xyz) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P06(2,1,0;xyz) P08(0,0,0;zxy)	T1: P02(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P02(0,0,2;xyz) P05(1,0,0;xyz) P06(0,2,0;yxz) P07(0,1,0;xzy) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P02(1,2,0;xzy) P06(2,2,2;xyz) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy) P09(2,0,0;xyz)
	Objective Awal	488	541	557	13.088	14.449,333	12.115,333
	Objective Akhir	488	545	613	16.052	17.467,333	16.105,333
	Durasi (s)	0,061	0,151	0,192	0,128	0,219	0,262
	Iterasi	0	1	2	1	2	3

TC3	Initial State	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P06(1,0,0;yxz) P16(2,1,0;xzy)	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P06(1,0,0;yxz) P16(2,1,0;xzy)	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)
	Final State	T1: P07(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P19(2,2,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P16(2,1,0;zyx) P29(2,0,0;zyx) P30(0,3,0;xyz) T3: P03(1,1,1;xzy) P24(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P01(0,1,0;zyx) P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) P30(1,3,0;xyz) T3: P02(1,0,0;yxz) P16(2,1,0;xzy)	T1: P02(0,3,0;xyz) P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P19(2,0,0;zxy) P20(3,2,1;xyz) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) P29(3,3,0;xyz) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)	T1: P09(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P19(2,2,0;xzy) P20(1,1,0;zyx) P28(4,1,2;xyz) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) P30(3,2,0;xyz) T3: P01(1,0,0;xzy) P03(1,1,1;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P01(0,1,0;zyx) P05(3,0,0;zyx) P11(2,2,0;xzy) P20(0,0,0;zyx) P23(1,2,0;yzx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P10(0,1,0;xyz) P16(2,1,0;xzy) P27(1,0,0;yxz)	T1: P01(4,1,0;zxy) P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P15(3,0,0;xyz) P19(0,1,1;zyx) P21(3,0,2;zyx) P29(1,1,1;xyz) T3: P17(3,1,0;zxy) P25(2,0,0;xzy) P27(0,1,0;zyx)
	Objective Awal	846	904	789	17.696	20.198,762	20.098,048
	Objective Akhir	1.025	986	1.015	22.608	22.658,476	23.067,143
	Durasi (s)	2,990	1,857	2,269	2,456	2,018	2,146
	Iterasi	6	3	4	5	4	5

5.1.2 Sideways Move Hill Climbing

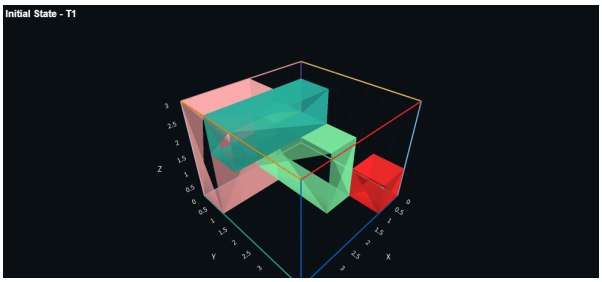
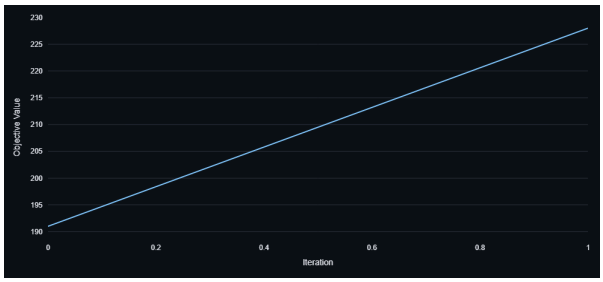
Objective :		Value			ETA Weighted		
Seeds :		42	43	44	42	43	44
TC1	Initial State	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)
	Final State	T1: P04(1,0,2;yxz) P05(3,3,0;xzy) P06(0,3,2;xzy) P08(2,1,0;xyz) P09(0,0,0;xzy)	T1: P04(1,0,1;xyz) P05(3,2,2;zyx) P06(0,1,0;zxy) P08(1,0,0;yzx) P09(2,3,0;zxy)	T1: P01(2,0,0;zyx) P05(3,2,2;zxy) P06(0,3,0;zyx) P09(1,1,0;zyx)	T1: P04(1,0,2;yxz) P05(3,3,0;xzy) P06(0,3,2;xzy) P08(2,1,0;xyz) P09(0,0,0;xzy)	T1: P04(1,0,1;xyz) P05(3,2,2;zyx) P06(0,1,0;zxy) P08(1,0,0;yzx) P09(2,3,0;zxy)	T1: P04(1,1,0;zyx) P05(2,1,0;xzy) P06(0,0,0;zxy) P08(2,0,1;zyx) P09(2,0,2;xyz)
	Objective Awal	191	344	195	7.431	17.034	4.480,714
	Objective Akhir	344	344	293	17.034	17.034	17.034
	Durasi (s)	3,289	2,314	2,343	1,754	1,424	1,754
	Iterasi	133	100	108	133	100	104
	Max sideways move	100	100	100	100	100	100
TC2	Initial State	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)
	Final State	T1: P04(0,1,2;yxz) P08(0,1,0;yxz) P10(2,0,0;yxz) P14(2,2,0;xyz) T2: P05(3,2,0;xzy) P06(0,1,1;zxy)	T1: P01(0,2,0;zyx) P07(0,0,2;yxz) P08(2,0,0;zyx) P10(1,2,1;yzx) P11(1,1,0;zxy) T2: P06(0,0,0;yxz)	T1: P01(0,2,0;zxy) P04(1,3,0;yzx) P05(3,1,0;xyz) P09(2,2,0;yzx) P14(1,0,0;yxz) T2: P06(2,2,0;xzy)	T1: P02(2,1,1;zxy) P07(0,2,0;xyz) P10(3,1,0;xyz) P14(0,0,0;xyz) T2: P05(3,0,0;yxz) P08(0,1,0;xyz)	T1: P02(0,2,0;xzy) P06(3,0,0;yxz) P07(2,0,2;yxz) P09(0,1,2;xyz) P10(2,0,0;xzy) T2: P01(2,0,1;yxz)	T1: P06(0,1,0;yxz) P07(1,0,1;zxy) P09(2,2,0;zyx) P10(2,0,2;yzx) P11(1,0,0;zxy) P14(2,0,0;yzx) T2:

		P07(0,0,0;xyz)	P09(1,1,0;zxy) P14(2,1,0;xyz)	P08(0,0,0;yxz) P10(2,0,0;xzy)	P11(3,0,1;yxz)	P11(3,0,0;zxy) P14(0,0,0;yxz)	P01(0,0,0;xzy) P02(0,2,0;xzy)
	Objective Awal	488	541	557	13.088	14.449,333	12.115,333
	Objective Akhir	680	625	616	21.595,333	17.737,333	17.737,333
	Durasi (s)	7,605	10,130	6,172	6,181	5,779	8,449
	Iterasi	114	149	109	174	125	216
	Max sideways move	100	100	100	100	100	100
TC3	Initial State	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P06(1,0,0;yxz) P16(2,1,0;xzy)	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P06(1,0,0;yxz) P16(2,1,0;xzy)	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)
	Final State	T1: P14(0,1,0;zyx) P15(2,1,0;xzy) P19(3,2,0;zxy) P20(1,3,0;xyz) P27(1,1,0;yxz) T2: P02(2,3,0;zyx) P11(2,1,0;zxy) P16(0,0,1;xzy) P29(1,1,0;yzx) P30(2,1,2;zyx) T3: P03(1,0,0;xyz) P24(1,1,1;xzy) P26(1,1,0;yzx)	T1: P03(2,0,0;xyz) P09(0,3,0;yzx) P10(3,1,1;zyx) P15(0,0,2;zyx) P19(2,2,0;yxz) P27(0,2,0;xyz) T2: P05(0,2,0;xyz) P08(1,1,0;zyx) P11(2,2,0;xzy) P20(1,0,0;zyx) P24(3,0,2;yxz) P26(1,1,2;zyx) P29(0,0,0;xyz) T3:	T1: P02(4,0,1;yxz) P05(4,1,0;xyz) P19(2,1,0;xyz) P20(0,1,1;yxz) P26(1,0,0;yxz) P27(0,2,0;zxy) P29(1,0,2;zyx) T2: P03(0,1,0;xzy) P09(3,0,0;yxz) P11(2,2,1;xyz) P15(0,0,0;zxy) P21(0,3,0;zyx) T3: P08(2,1,1;yxz)	T1: P09(4,2,0;yzx) P14(2,0,0;zyx) P15(0,2,0;xzy) P19(3,0,0;zyx) P23(2,0,2;zyx) P28(1,1,0;zyx) T2: P02(1,0,2;zxy) P11(0,2,0;xzy) P20(2,2,0;xzy) P27(2,1,0;zyx) P30(0,2,2;zyx) T3: P01(1,2,1;xyz) P03(0,0,1;xzy)	T1: P03(3,0,0;yzx) P07(4,0,0;zyx) P09(2,2,0;zyx) P15(0,0,0;yzx) P19(0,3,0;yzx) P28(1,0,1;xyz) P30(4,1,2;zyx) T2: P01(1,0,0;zxy) P05(2,0,0;xyz) P11(2,2,0;yxz) P20(0,0,1;xyz) P23(3,0,0;yzx) P24(3,0,2;zxy) P29(2,3,2;zxy)	T1: P08(3,0,0;xyz) P09(0,3,0;yzx) P16(0,0,0;xzy) P20(4,1,0;yzx) P22(1,2,0;xzy) T2: P03(1,0,0;zxy) P05(0,3,0;zxy) P15(3,1,0;xyz) P19(1,0,1;zyx) P29(1,3,0;zyx) T3: P07(1,0,0;xyz) P10(0,1,0;xzy) P27(2,2,0;zyx)

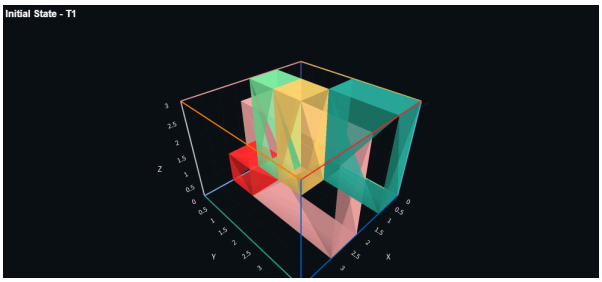
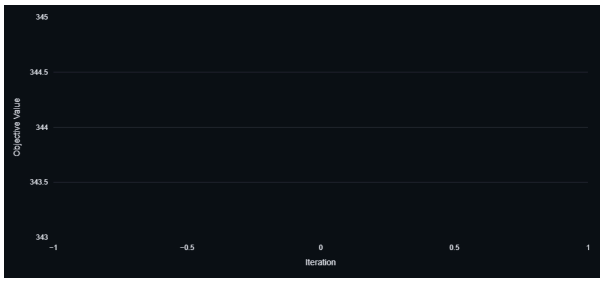
			P02(1,0,0;yxz) P16(2,1,0;xzy) P30(2,0,0;zxy)	P16(0,1,0;yxz) P25(3,0,0;zxy) P30(2,2,0;xzy)	P26(2,1,0;yzx)	T3: P10(2,0,0;yxz) P16(0,0,0;xyz) P27(2,1,0;yxz)	P30(3,1,0;xyz)
	Objective Awal	846	904	789	17.696	20.198,762	20.098,048
	Objective Akhir	1.029	1.050	1.048	22.608	23.472,476	24.141,571
	Durasi (s)	46,287	94,644	83,107	45,772	63,581	63,777
	Iterasi	110	205	186	105	180	237
	Max sideways move	100	100	100	100	100	100

1. Testcase 1 Small

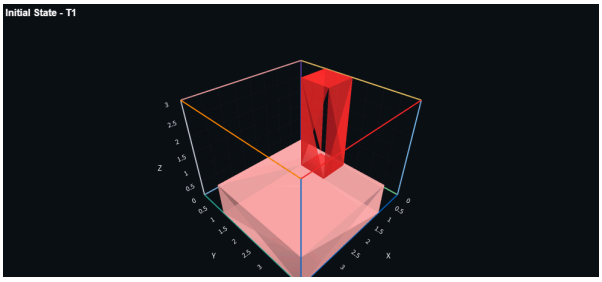
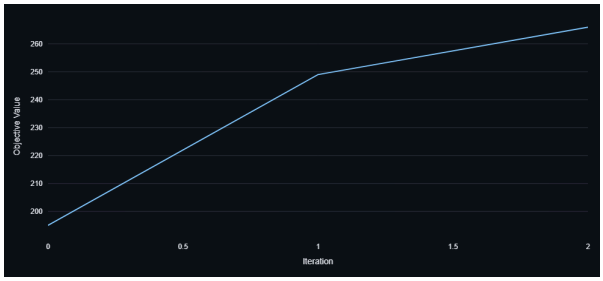
a. Run 1

Visualisasi Initial State	Graf Perkembangan Objective
	

b. Run 2

Visualisasi Initial State	Graf Perkembangan Objective
	

c. Run 3

Visualisasi Initial State	Graf Perkembangan Objective
	

5.1.3 Random Restart Hill Climbing

Objective :		Value			ETA Weighted		
Seeds :		42	43	44	42	43	44
TC1	Initial State	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)
	Final State	T1: P04(1,0,0;xzy) P05(1,2,0;zyx) P06(1,1,2;xyz) P08(1,1,1;zyx) P09(1,3,2;yxz)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P04(3,1,0;zyx) P05(0,3,0;xyz) P06(1,1,0;yzx) P08(2,2,0;xzy) P09(0,0,0;xzy)	T1: P04(3,0,0;zyx) P05(0,3,0;xyz) P06(2,3,0;zyx) P08(0,0,0;xyz) P09(2,0,0;xyz)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P04(1,1,0;yxz) P05(0,3,0;xyz) P06(0,0,0;xyz) P08(2,1,1;xyz) P09(0,1,0;zxy)
	Objective Awal	191	344	195	7.431	17.034	4.480,714
	Objective Akhir	344	344	344	17.034	17.034	17.034
	Durasi (s)	0,286	0,299	0,498	0,417	0,348	0,374
	Banyak Restart	10	10	10	10	10	10
	Iterasi /restart	[1, 2, 2, 1, 0, 1, 0, 2, 0, 2, 0]	[0, 4, 4, 0, 1, 0, 1, 1, 0, 3, 0]	[2, 1, 2, 1, 2, 1, 1, 2, 1, 2, 0]	[1, 2, 2, 3, 1, 1, 2, 2, 2, 1, 0]	[0, 2, 2, 1, 2, 1, 1, 0, 3, 0, 2]	[4, 4, 1, 3, 0, 3, 0, 1, 1, 0, 2]
	Maximum Restart	10	10	10	10	10	10
TC2	Initial State	T1: P04(1,1,2;xyz)	T1: P04(0,0,2;xyz)	T1: P01(0,2,0;zyx)	T1: P04(1,1,2;xyz)	T1: P04(0,0,2;xyz)	T1: P01(0,2,0;zyx)

		P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)	P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)
	Final State	T1: P04(0,0,0;zyx) P07(1,0,0;yxz) P10(3,1,0;yxz) P14(2,2,1;zyx) T2: P05(2,1,0;zyx) P06(2,0,0;xyz) P08(0,0,0;zxy)	T1: P02(3,0,0;zyx) P04(2,0,0;zyx) P05(0,3,2;xyz) P14(0,2,0;yxz) T2: P06(1,2,0;yzx) P08(1,0,0;xzy) P10(0,2,0;xyz)	T1: P04(1,0,0;yzx) P05(1,2,1;xyz) P13(0,1,0;xyz) T2: P06(2,1,0;xzy) P08(0,0,0;zxy) P10(3,0,0;xyz)	T1: P01(2,0,0;yzx) P02(1,1,0;xzy) P05(3,2,2;xyz) P07(0,2,0;xzy) P11(0,3,0;zyx) T2: P06(2,2,0;xyz) P08(0,0,0;zxy) P10(2,0,0;xyz)	T1: P02(0,0,0;zxy) P07(1,0,0;yxz) P08(1,0,1;xyz) T2: P01(2,1,0;zyx) P05(0,1,0;xyz) P06(3,1,0;zxy) P10(0,2,0;yxz) P11(0,0,0;xzy)	T1: P01(2,1,0;yzx) P02(1,1,0;zyx) P05(0,2,1;xyz) P08(2,2,0;zyx) P11(0,0,0;zxy) T2: P06(1,2,0;xyz) P07(0,0,1;xyz) P10(3,1,0;yzx)
	Objective Awal	488	541	557	13.088	14.449,333	12.115,333
	Objective Akhir	680	652	632	20.934	20.934	20.934
	Durasi (s)	1,142	1,648	1,861	2,093	1,838	1,850
	Banyak Restart	10	10	10	10	10	10
	Iterasi /restart	[0, 1, 3, 2, 1, 3, 1, 0, 3, 1, 3]	[1, 5, 5, 1, 1, 0, 1, 2, 2, 2, 3]	[2, 3, 4, 4, 2, 2, 2, 3, 0, 3, 2]	[1, 5, 0, 5, 2, 4, 3, 5, 1, 2, 2]	[2, 3, 2, 3, 5, 4, 3, 5, 0, 3, 3]	[3, 1, 5, 4, 5, 3, 4, 4, 3, 3, 1]
	Maximum Restart	10	10	10	10	10	10
TC3	Initial State	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3:	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3:	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)

			P06(1,0,0;yxz) P16(2,1,0;xzy)			P06(1,0,0;yxz) P16(2,1,0;xzy)	
	Final State	T1: P02(0,0,0;xyz) P05(4,0,0;xyz) P11(3,1,0;xyz) P15(0,2,0;xzy) P25(2,2,0;zyx) P29(2,2,2;zxy) T2: P16(0,1,0;xzy) P18(3,0,0;yxz) P19(0,2,2;yxz) P20(2,0,0;yxz) P30(1,0,0;xyz) T3: P03(0,1,1;xzy) P08(2,0,0;zyx) P09(1,2,0;xyz) P10(0,0,0;zxy)	T1: P02(1,2,0;xzy) P05(4,1,0;yzx) P10(0,0,0;yxz) P15(0,2,2;yzx) P20(3,2,1;yxz) P26(3,0,0;zxy) P27(1,1,1;xzy) P30(4,2,0;zyx) T2: P07(2,3,0;yzx) P14(0,1,0;xyz) P19(2,0,0;xzy) P29(2,2,0;zyx) T3: P11(2,1,0;yzx) P16(0,1,0;xzy)	T1: P03(4,0,0;yxz) P07(0,0,0;zxy) P10(1,2,0;xzy) P16(1,0,0;zyx) P25(2,2,0;yzx) P28(3,1,0;zxy) P29(2,2,2;zyx) P30(2,3,0;xyz) T2: P09(0,0,2;xzy) P11(1,1,0;yzx) P19(3,1,0;zxy) P21(0,2,0;zyx) P26(0,0,0;xzy) T3: P05(3,2,0;xyz) P15(0,0,0;zyx) P20(2,1,0;zyx) P24(0,0,1;yxz)	T1: P01(4,2,0;yzx) P03(0,2,0;xzy) P15(0,0,0;yzx) P19(0,1,1;yxz) P27(3,0,0;xzy) P29(0,1,2;zyx) T2: P09(0,3,0;xyz) P22(2,0,0;xyz) P23(1,1,0;yzx) P30(1,0,0;xyz) T3: P05(1,2,0;xzy) P07(3,0,0;zxy) P20(1,1,1;zxy) P26(1,0,0;zxy)	T1: P01(3,1,1;yxz) P03(2,0,0;xyz) P05(3,2,2;xyz) P08(3,3,0;xzy) P19(0,0,0;zyx) P26(1,2,0;xzy) P27(1,1,2;zxy) T2: P07(2,1,1;xzy) P09(3,1,0;yxz) P10(1,0,0;zxy) P15(0,2,0;xzy) P20(1,3,0;xyz) P21(2,2,0;yxz) P23(2,2,1;zxy) P30(3,3,1;yxz) T3: P11(0,0,0;xyz) P16(2,1,0;yxz)	T1: P02(0,0,1;xyz) P03(0,1,0;yxz) P05(0,3,2;yzx) P07(1,0,0;yxz) P09(1,2,0;zyx) P15(2,1,1;yxz) P20(3,2,0;xyz) T2: P16(1,1,0;xzy) P19(0,0,0;xzy) P26(3,0,0;xyz) P27(2,3,0;zyx) P30(1,1,2;yzx) T3: P11(1,1,0;xzy) P25(1,0,0;yzx)
	Objective Awal	846	904	789	17.696	20.198,762	20.098,048
	Objective Akhir	1.051	1.023	1.041	23.814,905	23.300,19	23.647,286
	Durasi (s)	34,214	24,492	21,527	24,328	26,573	19,924
	Banyak Restart	10	10	10	10	10	10
	Iterasi /restart	[6, 5, 8, 6, 5, 5, 7, 4, 6, 6, 5]	[3, 5, 5, 6, 10, 6, 8, 2, 3, 8, 7]	[4, 5, 2, 4, 6, 7, 6, 6, 6, 7, 5]	[5, 7, 7, 3, 7, 5, 8, 4, 4, 5, 7]	[4, 7, 7, 9, 7, 8, 3, 6, 7, 3, 6]	[5, 5, 3, 7, 4, 5, 5, 5, 4, 6, 5]
	Maximum Restart	10	10	10	10	10	10

5.1.4 Stochastic Hill Climbing

Objective :		**Value**			**ETA Weighted**		
Seeds :		42	43	44	42	43	44
TC1	Initial State	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)	T1: P02(2,0,0;yzx) P05(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P09(0,1,0;zxy)

	Final State	T1: P02(2,0,0;yzx) P06(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P03(1,1,0;yxz) P05(0,0,0;xyz) P06(0,1,0;zxy) P09(2,1,1;xyz)	T1: P02(2,0,0;yzx) P06(0,3,0;xyz) P08(1,0,0;xyz) P09(1,1,2;yzx)	T1: P04(2,1,0;zyx) P05(2,0,0;zxy) P06(1,0,0;yzx) P08(0,2,0;xzy) P09(1,1,0;xzy)	T1: P04(1,1,0;yxz) P05(0,0,0;xyz) P06(0,1,0;zxy) P08(2,1,1;xyz) P09(3,1,1;xyz)
	Objective Awal	191	344	195	7.431	17.034	4.480,714
	Objective Akhir	228	344	266	9.828	17.034	17.034
	Durasi (s)	0,141	0,121	0,132	0,146	0,096	0,187
	Iterasi	1000	1000	1000	1000	1000	1000
TC2	Initial State	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P07(0,1,0;xzy) P09(0,2,0;yxz) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P08(0,0,0;zxy)
	Final State	T1: P04(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P04(0,0,2;xyz) P05(1,0,0;xyz) P06(0,2,0;yxz) P07(0,1,0;xzy) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P04(1,2,0;xzy) P05(1,3,0;xyz) P06(3,1,0;yzx) P10(0,2,2;xzy) P14(0,0,0;yxz) T2: P08(0,0,0;zxy) P09(2,1,0;xyz)	T1: P02(1,1,2;xyz) P05(3,2,1;yxz) P07(2,0,0;yxz) P14(0,2,0;yzx) T2: P03(0,0,0;zyx)	T1: P02(0,0,2;xyz) P05(1,0,0;xyz) P06(0,2,0;yxz) P07(0,1,0;xzy) P10(2,0,0;yxz) T2: P01(2,0,1;yxz) P11(3,0,0;zxy) P14(0,0,0;zyx)	T1: P01(0,2,0;zyx) P08(1,2,0;xzy) P10(0,2,2;xzy) P11(3,1,0;yzx) P14(0,0,0;yxz) T2: P02(0,0,0;zxy) P05(1,0,0;xyz) P06(1,1,0;xyz)
	Objective Awal	488	541	557	13.088	14.449,333	12.115,333
	Objective Akhir	488	545	616	16.052	17.467,333	15.835,333
	Durasi (s)	0,177	0,203	0,209	0,170	0,228	0,183
	Iterasi	1000	1000	1000	1000	1000	1000
TC3	Initial State	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy)	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy)	T1: P06(2,2,0;xzy) P12(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P20(1,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy)	T1: P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P24(4,1,0;zxy)

		T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P06(1,0,0;yxz) P16(2,1,0;xzy)	P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)	T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P23(2,0,0;zyx) P27(2,1,0;zyx) T3: P03(1,1,1;xzy) P17(1,0,0;xzy) P26(0,1,0;zyx)	P28(4,0,0;xyz) T2: P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P13(0,1,0;zyx) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P06(1,0,0;yxz) P16(2,1,0;xzy)	P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P11(0,1,1;zyx) P15(3,0,0;xyz) P21(3,0,2;zyx) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy)
	Final State	T1: P07(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P19(2,2,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P11(0,0,0;zxy) P16(2,1,0;zyx) P29(2,0,0;zyx) P30(3,3,0;xyz) T3: P03(1,1,1;xzy) P24(1,0,0;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P28(4,0,0;xyz) T2: P01(0,1,0;zyx) P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P20(0,0,0;zyx) P24(2,1,0;yzx) P29(2,0,0;yxz) T3: P02(1,0,0;yxz) P16(2,1,0;xzy) P30(0,1,0;xyz)	T1: P01(1,2,2;xzy) P18(1,0,0;yzx) P19(2,0,0;zxy) P20(3,0,0;xyz) P24(4,1,0;zxy) P26(1,2,0;xzy) P29(0,1,0;yzx) T2: P02(0,0,0;zxy) P05(3,0,2;zyx) P11(0,1,1;zyx) P15(3,0,0;xyz) T3: P08(3,1,0;zxy) P16(0,1,0;zyx) P25(2,0,0;xzy) P30(2,1,0;xyz)	T1: P07(0,1,0;zyx) P14(4,1,0;zyx) P15(3,0,0;xzy) P19(2,2,0;xzy) P20(1,1,0;zyx) T2: P02(0,1,2;xzy) P05(3,3,0;xyz) P09(2,0,0;zyx) P11(0,0,0;zxy) P27(2,1,0;zyx) T3: P01(1,0,0;xzy) P03(1,1,1;xzy) P26(0,1,0;zyx)	T1: P03(1,0,0;zyx) P07(0,1,0;yzx) P09(3,2,0;yzx) P18(0,3,0;zyx) P19(0,2,0;xzy) P23(4,0,0;xyz) T2: P01(2,1,0;yzx) P05(3,0,0;zyx) P08(1,2,0;yzx) P11(2,2,0;xzy) P20(0,0,0;zyx) P24(0,1,0;zyx) P29(2,0,0;yxz) P30(0,2,0;xyz) T3: P10(0,1,0;xyz) P16(2,1,0;xzy) P27(1,0,0;yxz)	T1: P01(4,1,0;zxy) P05(1,2,2;xzy) P07(1,2,0;xzy) P09(0,1,0;yzx) P22(2,0,0;zxy) P30(1,0,0;yzx) T2: P03(0,0,0;zxy) P15(3,0,0;xyz) P19(0,1,1;zyx) P28(3,0,2;zyx) T3: P17(3,1,0;zxy) P25(2,0,0;xzy) P27(0,1,0;zyx)
	Objective Awal	846	904	789	17.696	20.198,762	20.098,048
	Objective Akhir	1.025	986	1.028	22.859	22.581,048	22.949
	Durasi (s)	0,325	0,282	0,278	0,287	0,348	0,312
	Iterasi	1000	1000	1000	1000	1000	1000

5.2 Simulated Annealing

Objective :	**Value**			**ETA Weighted**		
Seeds :	1	2	3	1	2	3

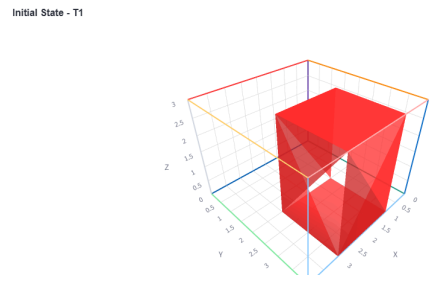
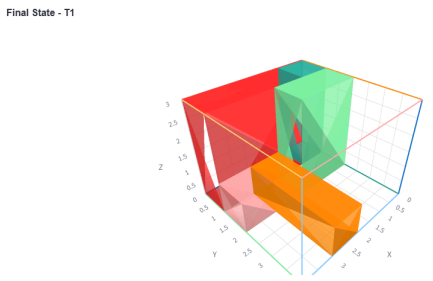
TC1	Initial State	T1: P07 (1,2,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P02 (0,2,0), P04 (3,0,0), P05 (2,3,1), P09 (0,1,0), P10 (2,1,0)	T1: P07 (1,2,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P02 (0,2,0), P04 (3,0,0), P05 (2,3,1), P09 (0,1,0), P10 (2,1,0)
	Final State	T1: P04 (1,0,0), P05 (3,1,0), P06 (0,0,0), P08 (0,1,0), P09 (2,1,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P04 (1,0,0), P05 (0,1,0), P06 (3,1,0), P08 (1,1,0), P09 (2,3,0)	T1: P04 (1,3,0), P05 (3,2,0), P06 (0,1,1), P08 (0,0,0), P09 (1,2,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P04 (3,0,0), P05 (0,3,0), P06 (2,1,0), P08 (0,0,0), P09 (2,0,0)
	Objective Awal	168	344	299	6.468	50.414	35.249
	Objective Akhir	344	344	344	50.414	50.414	50.414
	Durasi (s)	0,345	0,338	0,338	0,350	0,433	0,376
	Iterasi	2.000	2.000	2.000	2.000	2.000	2.000
	Accepted Worse	29	21	11	0	0	0
	Rejected	209	175	144	188	173	208
	Stuck Frequency	2	1	2	1	1	1
TC2	Initial State	T1: P01 (1,2,0), P02 (0,3,0), P04 (0,1,2), P06 (1,0,0); T2: P05 (1,2,0), P10 (2,2,0), P11 (1,1,0)	T1: P05 (2,0,1), P07 (0,0,0), P13 (0,3,0); T2: P06 (2,0,0), P09 (2,2,0), P10 (1,0,0), P11 (0,0,0)	T1: P01 (1,0,0), P13 (0,3,0), P14 (2,1,0); T2: P05 (0,0,0), P06 (3,2,0), P07 (0,1,0)	T1: P01 (1,2,0), P02 (0,3,0), P04 (0,1,2), P06 (1,0,0); T2: P05 (1,2,0), P10 (2,2,0), P11 (1,1,0)	T1: P05 (2,0,1), P07 (0,0,0), P13 (0,3,0); T2: P06 (2,0,0), P09 (2,2,0), P10 (1,0,0), P11 (0,0,0)	T1: P01 (1,0,0), P13 (0,3,0), P14 (2,1,0); T2: P05 (0,0,0), P06 (3,2,0), P07 (0,1,0)

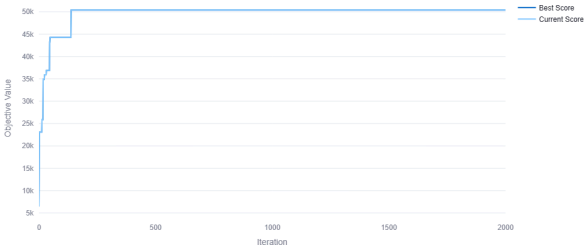
	Final State	T1: P04 (0,1,0), P05 (3,2,2), P08 (1,2,0), P14 (1,0,0); T2: P01 (3,0,0), P06 (0,0,0), P09 (2,1,0), P10 (0,2,0)	T1: P04 (0,0,0), P06 (3,0,0), P08 (0,2,0), P09 (2,3,0); T2: P01 (2,0,0), P05 (0,0,0), P10 (3,1,0), P14 (0,1,0)	T1: P02 (3,1,0), P07 (2,0,0), P08 (0,1,0); T2: P01 (2,0,1), P05 (3,1,0), P06 (0,1,0), P10 (1,1,0), P11 (0,0,0)	T1: P02 (1,0,1), P06 (0,3,0), P07 (0,0,0), P11 (1,3,0); T2: P01 (1,2,0), P05 (3,0,0), P10 (0,2,0), P14 (0,0,0)	T1: P02 (1,0,0), P05 (0,1,0), P06 (1,2,2), P07 (0,2,1), P10 (3,2,0), P11 (0,3,2); T2: P01 (0,1,0), P08 (1,1,0)	T1: P01 (0,0,0), P06 (0,3,0), P07 (3,0,1), P08 (0,0,1), P11 (1,3,0); T2: P02 (0,0,1), P05 (2,0,0), P10 (0,0,0)
	Objective Awal	413	489	524	32.888	37.742,57	38.252,57
	Objective Akhir	616	616	609	51.384	61.584	61.584
	Durasi (s)	0,434	0,362	0,396	0,444	0,528	0,567
	Iterasi	2.000	2.000	2.000	2.000	2.000	2.000
	Accepted Worse	21	16	30	2	0	2
	Rejected	145	161	195	202	195	235
	Stuck Frequency	4	3	3	2	2	2
TC3	Initial State	T1: P02 (2,3,1), P08 (2,2,1), P12 (1,2,0), P17 (0,0,0), P18 (3,1,0), P23 (3,0,1), P28 (2,0,2), P30 (4,0,0); T2: P01 (2,0,2), P09 (0,3,2), P11 (2,2,0), P13 (1,2,0), P20 (3,0,0),	T1: P04 (1,1,0), P05 (0,2,0); T2: P01 (3,0,0), P08 (3,2,0), P13 (2,2,0), P17 (0,0,0), P18 (0,2,0), P21 (1,3,0), P24 (1,3,1); T3: P11 (0,1,0), P25 (2,0,0)	T1: P01 (2,0,2), P02 (1,1,0), P09 (3,0,1), P14 (2,1,0), P15 (0,0,0), P29 (4,2,0), P30 (1,1,2); T2: P11 (1,0,0), P13 (0,2,0), P26 (0,2,2), P27 (2,2,0), P28 (1,1,2); T3: P06	T1: P02 (2,3,1), P08 (2,2,1), P12 (1,2,0), P17 (0,0,0), P18 (3,1,0), P23 (3,0,1), P28 (2,0,2),	T1: P04 (1,1,0), P05 (0,2,0); T2: P01 (3,0,0), P08 (3,2,0), P13 (2,2,0), P17 (0,0,0), P18 (0,2,0), P21 (1,3,0), P24 (1,3,1); T3: P11 (0,1,0), P25 (2,0,0)	T1: P01 (2,0,2), P02 (1,1,0), P09 (3,0,1), P14 (2,1,0), P15 (0,0,0), P29 (4,2,0), P30 (1,1,2); T2: P11 (1,0,0), P13 (0,2,0), P26 (0,2,2), P27 (2,2,0), P28 (1,1,2); T3: P06

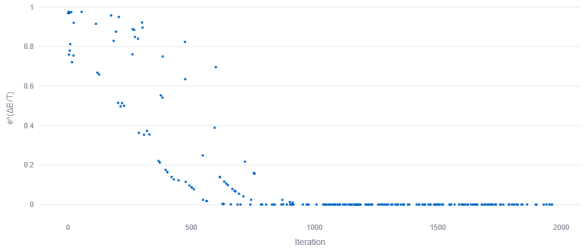
		P24 (0,2,0); T3: P06 (0,0,0), P07 (3,0,0)		(1,1,0), P07 (1,0,0)	P30 (4,0,0); T2: P01 (2,0,2), P09 (0,3,2), P11 (2,2,0), P13 (1,2,0), P20 (3,0,0), P24 (0,2,0); T3: P06 (0,0,0), P07 (3,0,0)		(1,1,0), P07 (1,0,0)
	Final State	T1: P01 (2,3,1), P03 (2,1,0), P08 (3,0,0), P11 (0,2,0), P20 (0,0,0), P24 (2,1,1), P28 (1,0,0), P29 (2,0,2); T2: P05 (2,0,0), P07 (3,2,0), P10 (0,0,1), P16 (1,2,0), P17 (0,1,0), P27 (1,1,2), P30 (0,2,1); T3: P09 (0,2,0), P21 (1,2,1), P25 (2,0,0), P26 (3,1,0)	T1: P03 (0,0,1), P09 (4,0,0), P19 (3,1,0), P21 (1,1,0), P25 (2,0,1), P27 (0,2,0), P28 (3,1,2); T2: P10 (0,3,0), P11 (2,1,1), P16 (0,0,0), P26 (2,1,0), P29 (0,0,2); T3: P05 (3,2,0), P08 (2,1,0), P17 (1,1,1), P20 (3,0,0), P24 (1,0,0), P30 (1,0,1)	T1: P02 (2,2,0), P05 (3,0,2), P09 (4,1,0), P16 (0,0,0), P20 (0,1,2), P21 (3,1,0), P23 (3,3,0), P24 (0,0,2), P26 (2,0,0); T2: P01 (1,0,0), P10 (2,0,0), P11 (0,1,0), P15 (3,1,0), P28 (1,1,2); T3: P08 (3,1,0), P17 (0,1,0), P25 (1,0,0), P29 (2,2,0), P30 (3,1,1)	T1: P05 (0,0,0), P07 (2,0,0), P09 (4,1,0), P15 (2,2,0), P19 (1,0,0), P21 (4,0,0), P23 (0,2,0), P26 (2,2,2); T2: P01 (3,1,2), P03 (0,2,0), P10 (1,0,0), P11 (2,2,0),	T1: P02 (0,1,0), P03 (2,1,0), P05 (4,2,0), P15 (1,0,0), P26 (4,2,1), P27 (2,1,1), P30 (0,3,2); T2: P01 (1,3,0), P07 (2,1,0), P09 (0,0,0), P19 (1,2,0), P20 (2,0,2), P23 (0,1,0); T3: P10 (0,0,0), P21 (1,1,0), P24 (1,0,1), P25 (2,0,0)	T1: P02 (0,0,0), P03 (1,0,2), P05 (4,2,0), P09 (3,1,1), P19 (2,0,0), P20 (4,1,1), P23 (3,2,0); T2: P01 (1,0,0), P07 (1,2,0), P10 (0,1,2), P16 (2,2,0), P26 (0,1,0), P28 (1,3,2); T3: P11 (0,1,0), P27 (3,1,0), P30 (2,1,0)

					P28 (0,1,0); T3: P17 (2,1,0), P20 (0,1,0), P27 (0,2,0), P29 (1,0,0), P30 (3,2,0)		
	Objective Awal	769	689	830	39.110,43	30.639	45.507,14
	Objective Akhir	965	991	987	67.372,29	66.285,86	67.548,43
	Durasi (s)	0,762	1,330	1,666	1,525	1,552	1,560
	Iterasi	2.000	2.000	2.000	2.000	2.000	2.000
	Accepted Worse	19	41	34	1	3	3
	Rejected	119	210	121	273	444	608
	Stuck Frequency	3	5	6	4	3	4

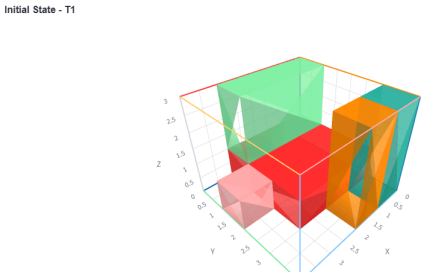
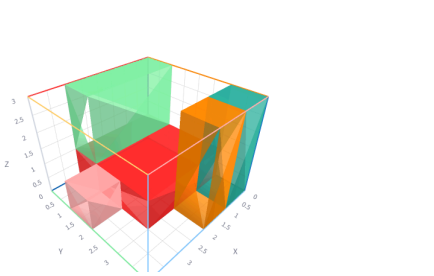
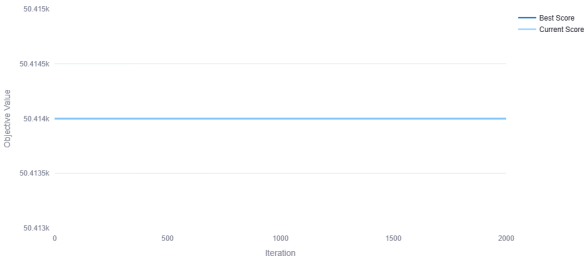
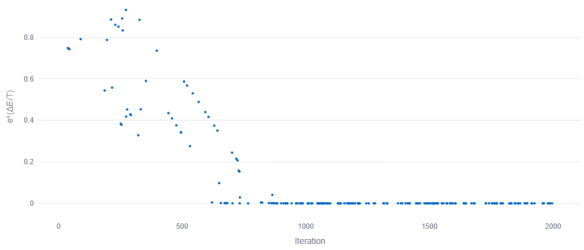
1. Test Case 1 - Small
   a. Run 1

Visualisasi Initial State	**Visualisasi Final State**
	
Graf Perkembangan Objective	**Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati**



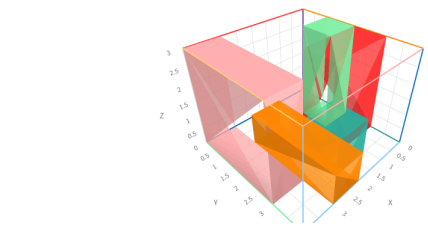
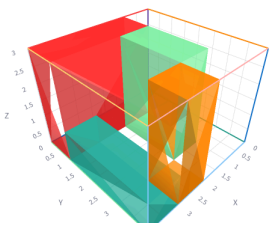
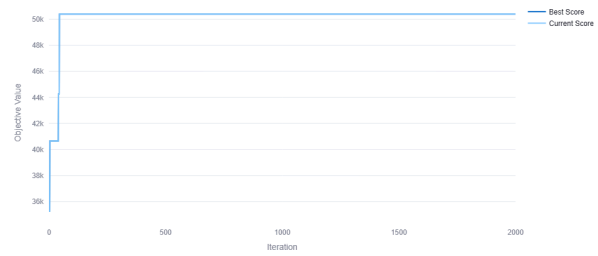
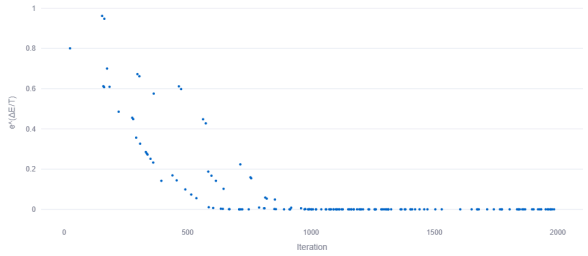


b. Run 2

Visualisasi Initial State	**Visualisasi Final State**
	
Graf Perkembangan Objective	**Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati**
	

c. Run 3

Visualisasi Initial State	**Visualisasi Final State**

	
Graf Perkembangan Objective	**Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati**
	

5.3 Genetic Algorithm

5.3.1 Pengaturan Eksperimen

Eksperimen GA dijalankan lewat frontend yang memanggil API, dan hasil tiap run disimpan otomatis sebagai berkas JSON dan plot PNG pada folder output/genetic_algorithm/. Seed yang dipakai adalah 1, 2, dan 3. Selain population_size dan max_iterations, parameter lain dibuat tetap pada semua run, yaitu tournament_size 2, mutation_rate 0,7, dan crossover_rate 0,8. Mode eta_weighted memakai eta_bonus 300.

Eksperimen dibagi menjadi tiga bagian. Bagian pertama menjalankan ketiga testcase dengan kedua objective pada populasi 50 dan 100 iterasi, masing-masing tiga seed (18 run). Bagian kedua (eksperimen A) menjadikan populasi 50 sebagai kontrol dan memvariasikan iterasi menjadi 50, 100, dan 200. Bagian ketiga (eksperimen B) menjadikan iterasi 100 sebagai kontrol dan memvariasikan populasi menjadi 20, 50, dan 100. Eksperimen A dan B dilakukan pada testcase medium dengan objective value, dan konfigurasi populasi 50 iterasi 100 dipakai bersama sebagai kontrol kedua eksperimen, sehingga totalnya 30 run.

Eksperimen parameter dilakukan pada test case medium (2 truk, 14 paket) karena ukurannya berada di tengah. Small terlalu mudah, skor awal sudah optimum di dua dari tiga seed, sehingga variasi parameter tidak terlihat pengaruhnya. Medium sudah memerlukan pencarian (ada paket yang harus ditinggalkan, rasio volume 1,26), masih multi-truk, dan hasilnya bervariasi antar seed dan konfigurasi. Large tidak dipilih untuk eksperimen parameter karena sudah mencakup 3 truk dan 30 paket pada eksperimen utama, dan tambahan variasi parameter di medium sudah cukup menunjukkan kecenderungan.

5.3.2 Hasil Populasi 50 dan Iterasi 100

Objective Awal adalah skor state awal dari run(), sama dengan Objective Awal pada tabel SA untuk seed yang sama. Skor Terbaik Populasi Awal adalah skor tertinggi di populasi awal, yang berisi state awal ditambah individu acak lain. Crossover dan Mutasi ditulis sebagai jumlah sukses dari jumlah percobaan, dengan sukses berarti hasilnya berbeda dari state asalnya. Visualisasi 3D hanya ditampilkan untuk testcase small dengan objective value sesuai kesepakatan kelompok, sedangkan plot seluruh run tersedia di folder output.

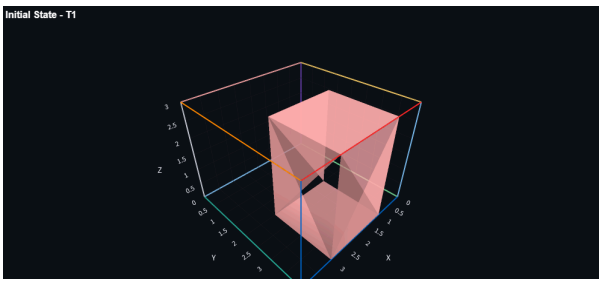
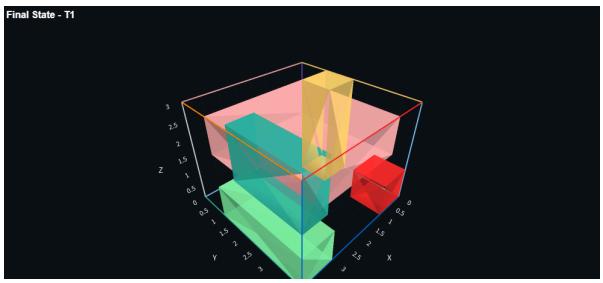
Objective :		**Value**			**ETA Weighted**		
Seeds :		1	2	3	1	2	3
TC1	Initial State	T1: P07 (1,2,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P02 (0,2,0), P04 (3,0,0), P05 (2,3,1), P09 (0,1,0), P10 (2,1,0)	T1: P07 (1,2,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P02 (0,2,0), P04 (3,0,0), P05 (2,3,1), P09 (0,1,0), P10 (2,1,0)
	Final State	T1: P04 (1,1,2), P05 (0,3,0), P06 (3,1,0), P08 (2,0,0), P09 (0,1,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P04 (1,0,0), P05 (0,0,0), P06 (2,1,0), P08 (1,2,0), P09 (0,1,0)	T1: P04 (1,1,2), P05 (0,3,0), P06 (3,1,0), P08 (2,0,0), P09 (0,1,0)	T1: P04 (0,0,0), P05 (3,1,0), P06 (0,3,0), P08 (0,0,1), P09 (1,3,0)	T1: P04 (2,1,0), P05 (3,1,0), P06 (3,2,0), P08 (0,0,0), P09 (0,3,0)
	Objective Awal	168	344	299	6.468	50.414	35.249
	Skor Terbaik Populasi Awal	344	344	336	50.414	50.414	45.048
	Objective Akhir	344	344	344	50.414	50.414	50.414

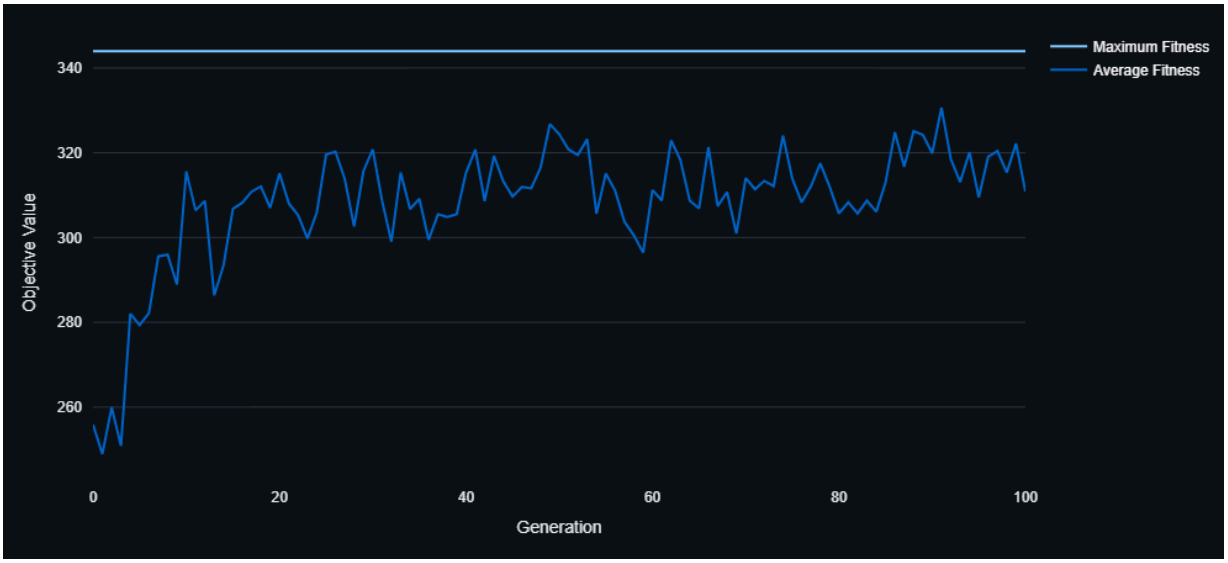
	Paket Termuat	5	5	5	5	5	5
	Durasi (s)	0,998	0,995	1,031	1,034	1,009	0,984
	Jumlah Populasi	50	50	50	50	50	50
	Iterasi	100	100	100	100	100	100
	Crossover (sukses / total)	2.754 / 3.903	2.799 / 3.936	2.774 / 4.008	2.676 / 3.936	2.819 / 3.979	2.770 / 3.896
	Mutasi (sukses / total)	3.173 / 3.414	3.236 / 3.437	3.277 / 3.445	3.126 / 3.401	3.287 / 3.477	3.264 / 3.443
TC2	Initial State	T1: P01 (1,2,0), P02 (0,3,0), P04 (0,1,2), P06 (1,0,0) T2: P05 (1,2,0), P10 (2,2,0), P11 (1,1,0)	T1: P05 (2,0,1), P07 (0,0,0), P13 (0,3,0) T2: P06 (2,0,0), P09 (2,2,0), P10 (1,0,0), P11 (0,0,0)	T1: P01 (1,0,0), P13 (0,3,0), P14 (2,1,0) T2: P05 (0,0,0), P06 (3,2,0), P07 (0,1,0)	T1: P01 (1,2,0), P02 (0,3,0), P04 (0,1,2), P06 (1,0,0) T2: P05 (1,2,0), P10 (2,2,0), P11 (1,1,0)	T1: P05 (2,0,1), P07 (0,0,0), P13 (0,3,0) T2: P06 (2,0,0), P09 (2,2,0), P10 (1,0,0), P11 (0,0,0)	T1: P01 (1,0,0), P13 (0,3,0), P14 (2,1,0) T2: P05 (0,0,0), P06 (3,2,0), P07 (0,1,0)
	Final State	T1: P04 (0,0,0), P06 (0,3,0), P07 (3,0,0), P14 (0,1,1) T2: P08 (1,0,0), P09 (0,2,0)	T1: P04 (3,0,0), P05 (0,3,0), P07 (1,0,0), P14 (0,1,1) T2: P06 (3,1,0), P08 (0,0,0), P10 (2,2,0)	T1: P01 (0,2,2), P04 (2,1,0), P06 (3,2,0), P08 (0,1,0), P10 (3,1,0) T2: P05 (2,2,0), P07 (0,0,0), P11 (0,0,1)	T1: P02 (1,0,0), P05 (1,0,1), P08 (0,1,1), P14 (2,1,1) T2: P06 (1,0,0), P07 (0,2,0), P10 (3,0,0)	T1: P01 (1,1,0), P07 (0,3,0), P08 (2,1,0), P10 (3,0,0), P11 (0,1,0) T2: P02 (1,0,0), P05 (0,2,0), P06 (0,0,0)	T1: P02 (2,3,0), P05 (0,3,0), P07 (0,2,0), P14 (0,0,0) T2: P06 (0,0,0), P08 (1,1,0), P10 (3,0,0)
	Objective Awal	413	489	524	32.888	37.742,57	38.252,57
	Skor Terbaik Populasi Awal	609	673	628	61.584	55.890	50.580
	Objective Akhir	664	680	645	62.544	61.584	62.544
	Paket Termuat	6	7	8	7	8	7
	Durasi (s)	1,980	1,933	1,941	2,067	1,905	1,772
	Jumlah Populasi	50	50	50	50	50	50
	Iterasi	100	100	100	100	100	100

	Crossover (sukses / total)	3.042 / 3.946	3.007 / 3.950	3.278 / 3.932	3.162 / 3.891	2.980 / 3.933	3.151 / 3.988
	Mutasi (sukses / total)	3.058 / 3.445	3.190 / 3.400	3.180 / 3.378	3.166 / 3.464	3.188 / 3.388	3.182 / 3.390
TC3	Initial State	T1: P02 (2,3,1), P08 (2,2,1), P12 (1,2,0), P17 (0,0,0), P18 (3,1,0), P23 (3,0,1), P28 (2,0,2), P30 (4,0,0) T2: P01 (2,0,2), P09 (0,3,2), P11 (2,2,0), P13 (1,2,0), P20 (3,0,0), P24 (0,2,0) T3: P06 (0,0,0), P07 (3,0,0)	T1: P04 (1,1,0), P05 (0,2,0) T2: P01 (3,0,0), P08 (3,2,0), P13 (2,2,0), P17 (0,0,0), P18 (0,2,0), P21 (1,3,0), P24 (1,3,1) T3: P11 (0,1,0), P25 (2,0,0)	T1: P01 (2,0,2), P02 (1,1,0), P09 (3,0,1), P14 (2,1,0), P15 (0,0,0), P29 (4,2,0), P30 (1,1,2) T2: P11 (1,0,0), P13 (0,2,0), P26 (0,2,2), P27 (2,2,0), P28 (1,1,2) T3: P06 (1,1,0), P07 (1,0,0)	T1: P02 (2,3,1), P08 (2,2,1), P12 (1,2,0), P17 (0,0,0), P18 (3,1,0), P23 (3,0,1), P28 (2,0,2), P30 (4,0,0) T2: P01 (2,0,2), P09 (0,3,2), P11 (2,2,0), P13 (1,2,0), P20 (3,0,0), P24 (0,2,0) T3: P06 (0,0,0), P07 (3,0,0)	T1: P04 (1,1,0), P05 (0,2,0) T2: P01 (3,0,0), P08 (3,2,0), P13 (2,2,0), P17 (0,0,0), P18 (0,2,0), P21 (1,3,0), P24 (1,3,1) T3: P11 (0,1,0), P25 (2,0,0)	T1: P01 (2,0,2), P02 (1,1,0), P09 (3,0,1), P14 (2,1,0), P15 (0,0,0), P29 (4,2,0), P30 (1,1,2) T2: P11 (1,0,0), P13 (0,2,0), P26 (0,2,2), P27 (2,2,0), P28 (1,1,2) T3: P06 (1,1,0), P07 (1,0,0)
	Final State	T1: P02 (2,2,0), P11 (3,1,0), P18 (0,0,0), P24 (3,0,0), P26 (1,0,0), P30 (1,3,0) T2: P01 (2,2,0), P03	T1: P03 (3,2,0), P05 (1,3,0), P09 (4,3,0), P15 (4,1,0), P16 (0,0,0), P19 (0,2,0), P21 (2,2,0)	T1: P01 (3,2,0), P02 (0,0,0), P08 (4,1,0), P09 (4,2,0), P11 (0,2,0), P16 (1,0,0), P29 (3,0,0)	T1: P07 (3,0,0), P09 (3,1,0), P10 (0,2,0), P19 (4,1,0), P25	T1: P07 (4,0,0), P20 (2,1,0), P22 (3,2,0), P27 (1,0,0) T2: P05 (3,2,0), P08 (0,0,0), P15 (2,1,0), P16	T1: P07 (0,1,0), P09 (2,2,2), P10 (1,2,0), P11 (2,2,0), P16 (1,0,0), P26 (4,2,0), P29 (3,0,0), P30 (2,3,2)

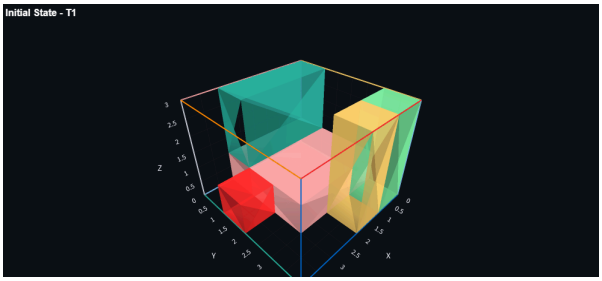
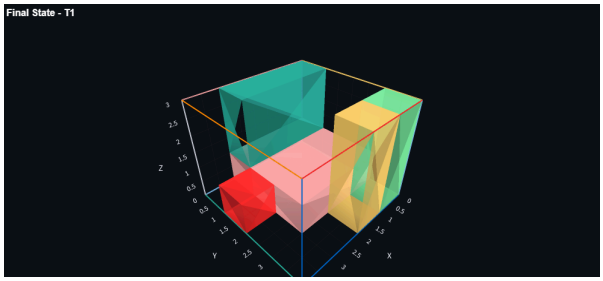
		(1,0,0), P15 (3,0,0), P20 (2,0,0), P25 (0,1,0) T3: P05 (0,2,1), P10 (3,1,0), P16 (0,0,0), P21 (0,2,0), P29 (2,0,0)	T2: P02 (0,2,0), P11 (2,0,0), P25 (3,2,0), P28 (2,3,0) T3: P18 (0,0,0), P20 (2,0,0), P26 (1,2,0)	T2: P19 (0,0,0), P24 (3,0,0), P26 (1,3,0), P27 (1,1,0), P28 (0,1,0), P30 (3,3,0) T3: P05 (0,2,0), P10 (3,0,0), P15 (0,0,0), P20 (1,2,0)	(1,3,0), P27 (1,1,0), P29 (1,0,0) T2: P20 (3,0,0), P22 (0,1,0), P23 (2,3,0) T3: P03 (0,2,0), P05 (0,0,0), P15 (1,0,0)	(0,2,0), P19 (1,0,0), P30 (3,1,0) T3: P03 (1,0,0), P09 (3,0,0), P10 (0,1,0), P29 (0,0,0)	T2: P05 (2,3,0), P18 (3,0,0), P19 (0,0,0), P20 (1,1,1), P27 (1,1,0) T3: P03 (1,2,0), P15 (0,0,0), P21 (0,2,0)
	Objective Awal	769	689	830	39.110,43	30.639	45.507,14
	Skor Terbaik Populasi Awal	953	932	975	64.383	60.127,57	64.266,43
	Objective Akhir	1.043	1.050	1.032	69.853	70.778,71	68.866,29
	Paket Termuat	16	14	17	13	14	16
	Durasi (s)	9,564	9,064	9,274	8,877	8,171	9,323
	Jumlah Populasi	50	50	50	50	50	50
	Iterasi	100	100	100	100	100	100
	Crossover (sukses / total)	3.647 / 3.876	3.749 / 3.939	3.629 / 3.856	3.570 / 3.932	3.510 / 3.912	3.485 / 3.841
	Mutasi (sukses / total)	3.290 / 3.447	3.257 / 3.411	3.324 / 3.467	3.217 / 3.455	3.183 / 3.392	3.231 / 3.442

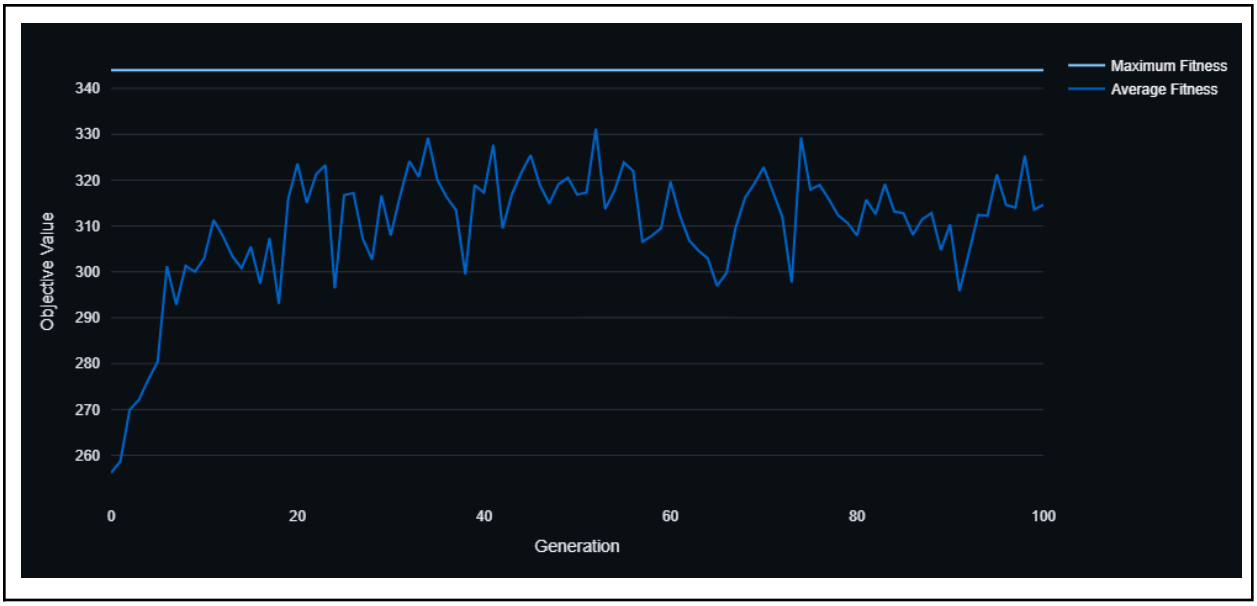
1. Test Case 1 - Small
    a. Run 1

Visualisasi Initial State	**Visualisasi Final State**
	

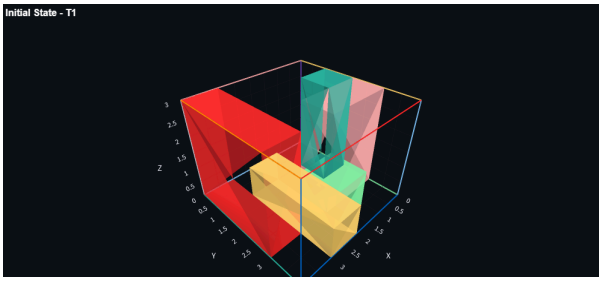
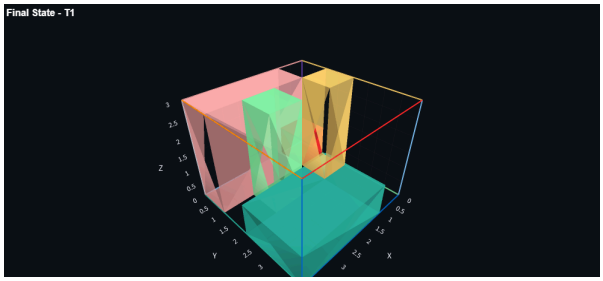
Graf Perkembangan Objective


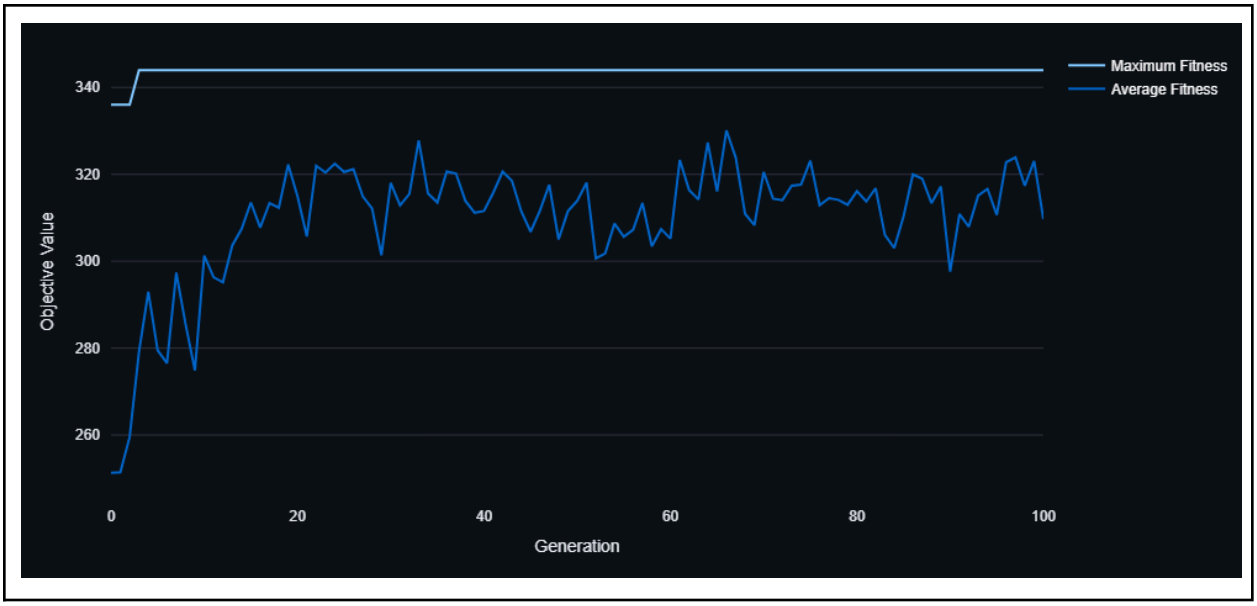
b. Run 2

Visualisasi Initial State	Visualisasi Final State
	
Graf Perkembangan Objective	



c. Run 3

Visualisasi Initial State	Visualisasi Final State
	
Graf Perkembangan Objective	

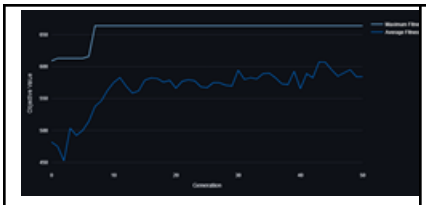
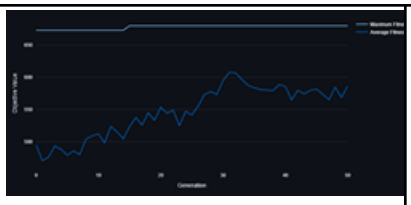
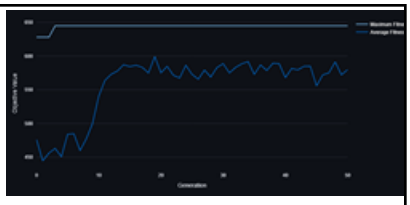
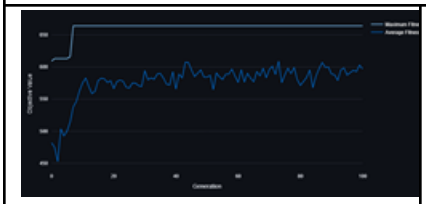
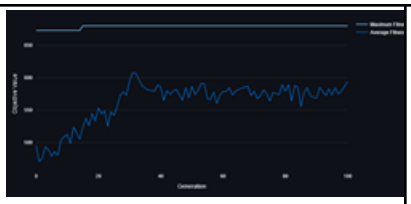
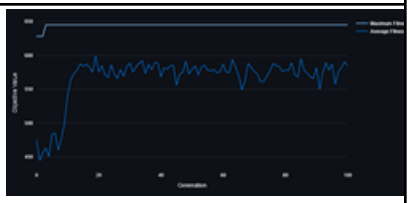
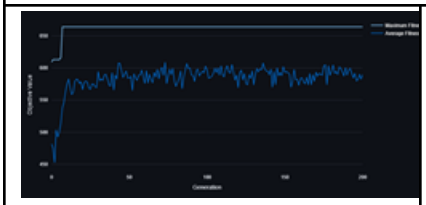
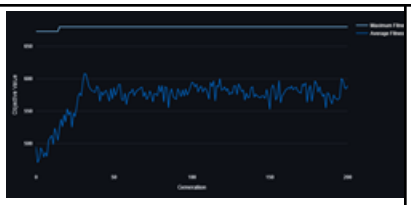
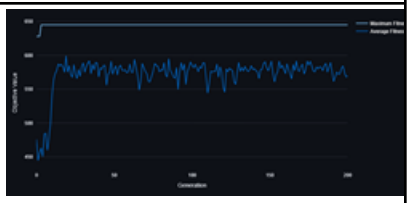


5.3.3 Eksperimen A: Variasi Iterasi (populasi 50, medium, value)

Populasi	Iterasi	Seed	Objective Awal	Skor Terbaik Populasi Awal	Objective Akhir	Durasi (s)	Crossover (sukses / total)	Mutasi (sukses / total)
50	50	1	413	609	664	1,347	1.571 / 1.964	1.555 / 1.728
50	50	2	489	673	680	1,323	1.521 / 1.971	1.593 / 1.707
50	50	3	524	628	645	1,248	1.640 / 1.968	1.587 / 1.689
Rata-rata (populasi 50, iterasi 50)					**663,0**	**1,306**		
50	100	1	413	609	664	1,980	3.042 / 3.946	3.058 / 3.445
50	100	2	489	673	680	1,933	3.007 / 3.950	3.190 / 3.400
50	100	3	524	628	645	1,941	3.278 / 3.932	3.180 / 3.378
Rata-rata (populasi 50, iterasi 100)					**663,0**	**1,951**		
50	200	1	413	609	664	3,319	5.831 / 7.924	6.067 / 6.823
50	200	2	489	673	680	3,233	6.206 / 7.866	6.400 / 6.799
50	200	3	524	628	645	3,195	6.570 / 7.852	6.449 / 6.796
Rata-rata (populasi 50, iterasi 200)					**663,0**	**3,249**		

Plot skor maksimum dan rata-rata populasi

Populasi 50, Iterasi 50

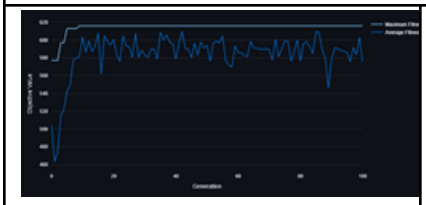
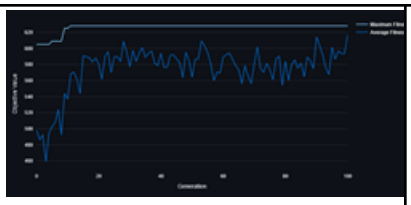
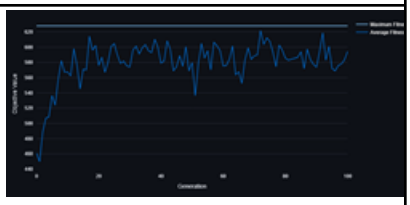
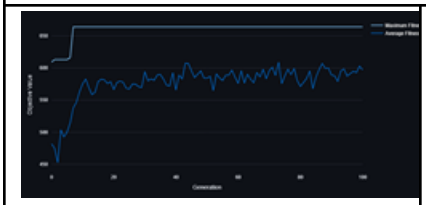
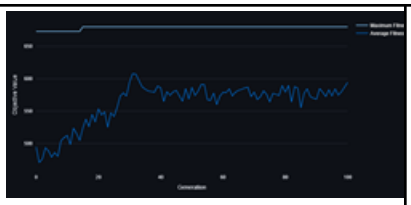
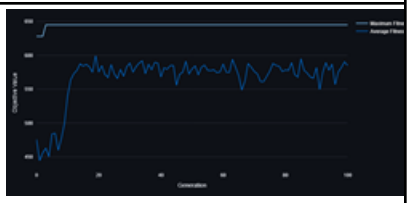
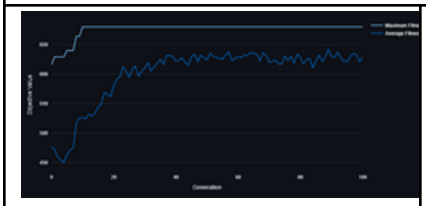
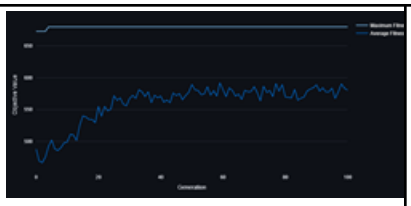
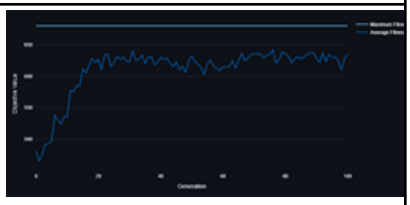
		
seed 1	seed 2	seed 3
Populasi 50, Iterasi 100		
		
seed 1	seed 2	seed 3
Populasi 50, Iterasi 200		
		
seed 1	seed 2	seed 3

5.3.4 Eksperimen B: Variasi Populasi (iterasi 100, medium, value)

Populasi	Iterasi	Seed	Objective Awal	Skor Terbaik Populasi Awal	Objective Akhir	Durasi (s)	Crossover (sukses / total)	Mutasi (sukses / total)
20	100	1	413	577	616	0,997	989 / 1.515	1.258 / 1.341
20	100	2	489	605	628	0,722	898 / 1.534	1.266 / 1.360
20	100	3	524	628	628	0,626	973 / 1.523	1.256 / 1.341
Rata-rata (populasi 20, iterasi 100)					**624,0**	**0,781**		
50	100	1	413	609	664	1,980	3.042 / 3.946	3.058 / 3.445
50	100	2	489	673	680	1,933	3.007 / 3.950	3.190 / 3.400
50	100	3	524	628	645	1,941	3.278 / 3.932	3.180 / 3.378
Rata-rata (populasi 50, iterasi 100)					**663,0**	**1,951**		
100	100	1	413	616	680	3,666	6.600 / 7.913	6.469 / 6.936
100	100	2	489	673	680	3,903	7.037 / 7.934	6.607 / 6.955
100	100	3	524	680	680	3,555	6.618 / 7.965	6.459 / 6.903

Rata-rata (populasi 100, iterasi 100)	**680,0**	**3,708**	

Plot skor maksimum dan rata-rata populasi

Populasi 20, Iterasi 100		
		
seed 1	seed 2	seed 3
Populasi 50, Iterasi 100		
		
seed 1	seed 2	seed 3
Populasi 100, Iterasi 100		
		
seed 1	seed 2	seed 3

BAB VI
ANALISIS

6.1 Kedekatan ke Lokal Optima

6.1.1 Hill Climbing

a. Steepest Ascent

Steepest Ascent memilih neighbor dengan objective tertinggi dan hanya menerima peningkatan skor. Tabel membandingkan hasil tiga seed dengan nilai acuan script Python pembanding. Pengujian menggunakan maksimum 1000 iterasi; ETA Weighted menggunakan eta_bonus = 100.

Testcase	Objective	Optimum	Seed 42	Seed 43	Seed 44	Avg	% Optimum	Run
Small	Value	344	228	344	266	279,3	81,2%	1/3
Small	ETA	50.414	9.828,0	17.034,0	17.034,0	14.632,0	29,0%	0/3
Medium	Value	680	488	545	613	548,7	80,7%	0/3
Medium	ETA	64.436	16.052,0	17.467,3	16.105,3	16.541,6	25,7%	0/3
Large	Value	1.082	1.025	986	1.015	1.008,7	93,2%	0/3
Large	ETA	71.363,2	22.608,0	22.658,5	23.067,1	22.777,9	31,9%	0/3

Analisis:

- Small: Value menghasilkan skor 228, 344, dan 266, dengan rata-rata 279,3 atau 81,2% acuan; hanya seed 43 mencapai 344. Pada ETA, skor akhir ketiga seed adalah 9.828,0, 17.034,0, dan 17.034,0, dengan rata-rata 14.632,0 atau 29,0% acuan. Perbedaan hasil antar-seed menunjukkan pengaruh state awal dan jalur pencarian.
- Medium: Value mencapai rata-rata 548,7 atau 80,7% acuan, tanpa run mencapai 680. ETA mencapai rata-rata 16.541,6 atau 25,7% acuan; hasil terbaik diperoleh seed 43 sebesar 17.467,3.
- Large: Value mencapai rata-rata 1.008,7 atau 93,2% acuan, tanpa run mencapai 1.082. ETA mencapai rata-rata 22.777,9 atau 31,9% acuan; hasil terbaik diperoleh seed 44 sebesar 23.067,1.
- Optimum lokal: Seluruh hasil yang tersedia berhenti sebelum batas 1000 iterasi. Berdasarkan kondisi penghentian implementasi, tidak ditemukan neighbor yang lebih baik. State akhir merupakan optimum lokal nonketat terhadap

neighborhood yang digunakan, tetapi belum tentu optimum global. Algoritma tidak menerima langkah berskor sama atau lebih rendah sehingga dapat tertahan pada plateau.

- Interpretasi acuan ETA: Persentase ETA merupakan perbandingan numerik dengan acuan script pembanding. Kesamaan testcase, rumus objective, dan eta_bonus perlu dipastikan sebelum selisih skor ditafsirkan sebagai kualitas pencarian.

b. Sideways Move

Sideways move memungkinkan pencarian berpindah ke state dengan skor sama untuk mencari peluang peningkatan. Tabel berikut membandingkan objective akhir tiga seed terhadap nilai optimum. Pengujian menggunakan maksimum 1000 iterasi dan 100 sideways move berturut-turut; mode ETA Weighted menggunakan eta_bonus = 100.

Testcase	Objective	Optimum	Seed 42	Seed 43	Seed 44	Avg	% Optimum	Run
Small	Value	344	344	344	293	327,0	95,1%	2/3
Small	ETA	50.414	17.034,0	17.034,0	17.034,0	17.034,0	33,8%	0/3
Medium	Value	680	680	625	616	640,3	94,2%	1/3
Medium	ETA	64.436	21.595,3	17.737,3	17.737,3	19.023,3	29,5%	0/3
Large	Value	1.082	1.029	1.050	1.048	1.042,3	96,3%	0/3
Large	ETA	71.363,2	22.608,0	23.472,5	24.141,6	23.407,3	32,8%	0/3

Analisis :

- Small: Mode Value mencapai rata-rata 327,0 atau 95,1% acuan, dengan 2 dari 3 run mencapai 344. Mode ETA konsisten pada 17.034,0, setara 33,8% acuan 50.414; tidak ada run yang mencapai acuan.
- Medium: Mode Value mencapai rata-rata 640,3 atau 94,2% acuan; hanya seed 42 mencapai 680. Mode ETA mencapai rata-rata 19.023,3 atau 29,5% acuan. Seed 42 menghasilkan skor tertinggi, yaitu 21.595,3, sementara dua seed lainnya memperoleh 17.737,3.
- Large: Mode Value mencapai rata-rata 1.042,3 atau 96,3% acuan, tetapi seluruh run masih di bawah 1.082. Mode ETA mencapai rata-rata 23.407,3 atau 32,8% acuan; skor tertinggi diperoleh seed 44 sebesar 24.141,6.

- Optimum lokal dan plateau: Seluruh run berhenti setelah 100 sideways move berturut-turut tanpa peningkatan. Pada evaluasi penghentian, tidak ditemukan neighbor dengan skor lebih tinggi. Kondisi ini menunjukkan optimum lokal nonketat terhadap neighborhood yang digunakan, tetapi belum menjamin bahwa seluruh plateau telah dieksplorasi atau optimum global tercapai.
- Interpretasi ETA: Persentase ETA merupakan perbandingan numerik terhadap acuan script pembanding. Eksperimen menggunakan eta_bonus = 100; sebelum selisih tersebut diartikan sebagai kelemahan pencarian, kesamaan testcase, rumus objective, dan eta_bonus pada script acuan perlu dipastikan.

c. Random Restart

Random Restart menjalankan Steepest Ascent dari beberapa state awal dan menyimpan solusi terbaik. Setiap run menggunakan maksimum 10 restart dan 1000 iterasi per pencarian. Mode ETA Weighted menggunakan eta_bonus = 100. Tabel membandingkan skor akhir terhadap acuan script Python pembanding.

Testcase	Objective	Optimum	Seed 42	Seed 43	Seed 44	Avg	% Optimum	Run
Small	Value	344	344	344	344	344,0	100,0%	3/3
Small	ETA	50.414	17.034,0	17.034,0	17.034,0	17.034,0	33,8%	0/3
Medium	Value	680	680	652	632	654,7	96,3%	1/3
Medium	ETA	64.436	20.934,0	20.934,0	20.934,0	20.934,0	32,5%	0/3
Large	Value	1.082	1.051	1.023	1.041	1.038,3	96,0%	0/3
Large	ETA	71.363,2	23.814,9	23.300,2	23.647,3	23.587,5	33,1%	0/3

Analisis :

- Small: Seluruh seed pada Value mencapai acuan 344, sehingga rata-rata dan persentase pencapaian adalah 344,0 dan 100%. Pada ETA, seluruh seed menghasilkan 17.034,0 atau 33,8% acuan. Konsistensi skor tidak berarti susunan akhir paket selalu sama.
- Medium: Value menghasilkan rata-rata 654,7 atau 96,3% acuan; hanya seed 42 mencapai 680. ETA menghasilkan skor identik 20.934,0 pada seluruh seed, setara 32,5% acuan. Restart menghasilkan skor akhir yang lebih konsisten pada ETA dibandingkan Value.
- Large: Value menghasilkan rata-rata 1.038,3 atau 96,0% acuan, dengan skor tertinggi 1.051 pada seed 42. ETA mencapai rata-rata 23.587,5 atau 33,1%

acuan, dengan skor tertinggi 23.814,9 pada seed 42. Tidak ada run yang mencapai acuan pada kedua mode.

- Optimum lokal dan restart: Seluruh run menjalankan 10 restart atau 11 pencarian termasuk pencarian awal. Jumlah perpindahan terbanyak dalam satu pencarian hanya 10, jauh di bawah batas 1000. Setiap pencarian berhenti saat tidak ada neighbor yang lebih baik; restart kemudian menjelajahi state awal lain. Solusi akhir adalah yang terbaik dari seluruh pencarian, bukan selalu hasil restart terakhir.
- Perbandingan dan interpretasi: Pada Value, rata-rata Random Restart lebih tinggi daripada Steepest Ascent untuk Small, Medium, dan Large (344,0 dibandingkan 279,3; 654,7 dibandingkan 548,7; serta 1.038,3 dibandingkan 1.008,7). Restart membantu mengurangi ketergantungan pada satu state awal, tetapi 10 restart belum menjamin optimum global. Persentase ETA merupakan perbandingan numerik; kesamaan rumus dan eta_bonus pada script acuan perlu dipastikan sebelum menafsirkan selisihnya sebagai kualitas solusi.

d. Stochastic Hill Climbing

Stochastic memilih kandidat neighbor secara acak dan menerima perpindahan hanya jika objective meningkat. Seluruh percobaan menggunakan 1000 iterasi; mode ETA Weighted menggunakan eta_bonus = 100. Tabel membandingkan objective akhir tiga seed terhadap acuan script Python pembanding.

Testcase	Objective	Optimum	Seed 42	Seed 43	Seed 44	Avg	% Optimum	Run
Small	Value	344	228	344	266	279,3	81,2%	1/3
Small	ETA	50.414	9.828,0	17.034,0	17.034,0	14.632,0	29,0%	0/3
Medium	Value	680	488	545	616	549,7	80,8%	0/3
Medium	ETA	64.436	16.052,0	17.467,3	15.835,3	16.451,6	25,5%	0/3
Large	Value	1.082	1.025	986	1.028	1.013,0	93,6%	0/3
Large	ETA	71.363,2	22.859,0	22.581,0	22.949,0	22.796,3	31,9%	0/3

Analisis :

- Small: Mode Value mencapai rata-rata 279,3 atau 81,2% acuan, dengan 1 dari 3 run mencapai 344. ETA mencapai rata-rata 14.632,0 atau 29,0% acuan. Hasil

akhir kedua mode sama dengan Steepest Ascent pada seed yang bersesuaian, meskipun jalur pencariannya dapat berbeda.

- Medium: Value mencapai rata-rata 549,7 atau 80,8% acuan, tanpa run mencapai 680. Seed 44 memperoleh 616, sedikit lebih tinggi daripada Steepest Ascent yang memperoleh 613. ETA mencapai rata-rata 16.451,6 atau 25,5% acuan, dengan skor tertinggi 17.467,3 pada seed 43.
- Large: Value mencapai rata-rata 1.013,0 atau 93,6% acuan; skor tertinggi adalah 1.028 pada seed 44. ETA mencapai rata-rata 22.796,3 atau 31,9% acuan; skor tertinggi adalah 22.949,0 pada seed 44. Tidak ada run yang mencapai acuan pada kedua mode.
- Optimum lokal dan penghentian: Seluruh run berhenti karena mencapai batas 1000 iterasi, sehingga kondisi berhenti tidak membuktikan bahwa state akhir merupakan optimum lokal. Sampling acak dapat melewatkan neighbor yang lebih baik. Karena langkah berskor sama atau lebih rendah ditolak, pencarian juga dapat tertahan pada plateau.
- Efektivitas iterasi dan acuan: Dari 1000 iterasi, perpindahan yang diterima hanya 0 sampai 13 per run. Iterasi mencakup percobaan sampling tanpa perpindahan, sehingga tidak sebanding langsung dengan jumlah iterasi Steepest Ascent yang menghitung perpindahan diterima. Persentase ETA adalah perbandingan numerik; kesamaan rumus dan eta_bonus pada script acuan perlu dipastikan sebelum selisihnya ditafsirkan sebagai kualitas solusi.

6.1.2 Simulated Annealing

SA mencapai optimum eksak pada semua run di testcase small, tetapi tidak ada satu pun run di medium dan large yang mencapainya. Rata-rata skor akhir pada kedua testcase itu berada di 90,2 sampai 94,0% dari optimum.

Testcase	Objective	Optimum	Seed 1	Seed 2	Seed 3	Avg	% Optimum	Run
small	value	344	344	344	344	344,0	100,0	3/3
small	ETA	50.414	50.414	50.414	50.414	50.414,0	100,0	3/3
medium	value	680	616	616	609	613,7	90,2	0/3
medium	ETA	64.436	51.384	61.584	61.584	58.184,0	90,3	0/3
large	value	1.082	965	991	987	981,0	90,7	0/3

large	ETA	71.363,2	67.372,29	66.285,86	67.548,43	67.068,9	94,0	0/3

- Testcase small mencapai optimum. Keenam run mencapai optimum eksak (344 pada value dan 50.414 pada ETA-weighted). Pada seed 2, state awal kebetulan sudah optimal, sehingga bukti kemampuan pencarian berasal dari seed 1 dan 3. Kedua seed itu menemukan skor akhirnya pada iterasi ke-45 sampai ke-370
- Testcase medium dan large tidak tercapai, selisih terhadap optimum berkisar 4,4–20,3% per run, dengan rata-rata 6,0 sampai 9,8% per kelompok. Pencilan terbesar adalah medium ETA-weighted seed 1: 51.384 (79,7% optimum), sedangkan seed 2 dan 3 mencapai 61.584 (95,6%).
- Medium dan large relatif setara, pada objective value, rata-ratanya 90,2% (medium) dan 90,7% (large). Dengan hanya tiga ukuran testcase, hubungan antara ukuran masalah dan kualitas hasil di atas small belum dapat disimpulkan.

Penyebabnya adalah terjebak di local optimum. Seluruh neighbor satu langkah dari 18 state akhir dienumerasi dengan fungsi neighbors() (swap, rotate, memindahkan ke semua koordinat, dan mengeluarkan paket). Tidak ada neighbor yang lebih baik pada satu pun state. Tiap state memiliki 30–172 neighbor valid, dan 74% di antaranya bernilai sama. Jadi 12 run yang di bawah optimum berhenti di local optimum, bukan kehabisan iterasi di tengah pendakian. Pada local optimum tidak ada neighbor yang lebih baik, sehingga keluar darinya membutuhkan langkah turun yang diterima SA secara probabilistik. Untuk lembah sedalam 72 poin pada medium, satu langkah –36 diterima dengan probabilitas 0,17 pada T = 20, tetapi hanya 0,0007 pada T = 5. Setelah T cukup rendah, lembah seperti ini praktis tidak dapat dilewati. SA mendekati optimum penuh pada ruang kecil (small), tetapi berhenti sekitar 7 sampai 10% di bawah optimum pada medium dan large. Penyebabnya local optimum yang jalan keluarnya membutuhkan beberapa langkah turun berurutan.

6.1.3 Genetic Algorithm

Nilai optimum yang dipakai sama dengan subbab 6.1.2, yaitu optimum global tiap testcase yang dihitung dengan skrip Python tersendiri. Persentase dihitung dari rata-rata skor akhir tiga run dibagi optimum, dikali 100.

Testcase	Objective	Optimum	Mean (3 seed)	% optimum	Run mencapai optimum
small	value	344	344,0	100,0	3/3
small	ETA	50.414	50.414,0	100,0	3/3
medium	value	680	663,0	97,5	1/3
medium	ETA	64.436	62.224,0	96,6	0/3
large	value	1.082	1.041,7	96,3	0/3
large	ETA	71.363,2	69.832,7	97,9	0/3

Pada testcase small, keenam run mencapai optimum. Namun pada empat dari enam run (seed 1 dan 2 untuk kedua objective), individu terbaik di populasi awal sudah bernilai sama dengan optimum, sehingga bukti kemampuan pencarian hanya berasal dari seed 3, yang naik dari 336 ke 344 pada value dan dari 45.048 ke 50.414 pada ETA.

Pada medium dan large, GA belum mencapai optimum kecuali satu run (medium value seed 2 yang mencapai 680). Rata-ratanya berada di 96,3 sampai 97,9% dari optimum, lebih tinggi daripada SA yang pada objective value hanya mencapai 90,2% (medium) dan 90,7% (large). Skor terbaik GA pada large value adalah 1.050 (seed 2), atau sekitar 97,0% dari optimum 1.082. Berbeda dengan SA, pada GA belum dilakukan enumerasi neighbor di final state, sehingga belum dapat dipastikan bahwa run yang di bawah optimum berhenti di local optimum. Dugaannya, operator mutasi dan crossover belum mampu menemukan susunan yang lebih baik dari state terbaik yang sudah dicapai, karena skor terbaik pada medium sudah tercapai pada iterasi 3 sampai 15 dan tidak berubah sampai iterasi akhir.

Perbandingan dengan SA perlu dibaca dengan hati-hati karena GA mengevaluasi lebih banyak state. Dengan populasi 50 dan 100 iterasi, GA mengevaluasi sekitar 50 × 101 = 5.050 state, sedangkan SA 2.000 iterasi mengevaluasi sekitar 2.000 state.

6.2 Dinamika Proses Pencarian

Hampir semua langkah SA bersifat netral (ΔE = 0) dan langkah yang menurunkan skor hanya sempat diterima di awal pencarian pada objective value, dan hampir tidak pernah diterima pada objective ETA-weighted.

6.2.1 Komposisi keputusan per iterasi

Berikut adalah jumlah keputusan per run dari 2.000 iterasi (rata-rata 3 seed). Tidak ada kegagalan pembangkitan neighbor pada seluruh run.

Testcase	Objective	Lebih baik	Setara	Lebih buruk (diterima)	Lebih buruk (ditolak)	Acceptance rate (%)
small	value	23,7	1.780,0	20,3	176,0	91,2
small	ETA	3,7	1.806,7	0,0	189,7	90,5
medium	value	25,0	1.785,7	22,3	167,0	91,6
medium	ETA	7,3	1.780,7	1,3	210,7	89,5
large	value	45,7	1.773,0	31,3	150,0	92,5
large	ETA	18,7	1.537,3	2,3	441,7	77,9

- Mayoritas langkah netral, neighbor setara (ΔE = 0) diterima pada 89,0% iterasi (value) dan 85,4% (ETA-weighted). Sebagian besar neighbor hanya menggeser atau memutar paket tanpa mengubah paket yang terangkut, sehingga SA banyak bergerak di dataran, mirip sideways move pada hill-climbing.
- Acceptance rate tinggi tetapi nilainya 77,9–92,5%, hampir seluruhnya berasal dari langkah netral, bukan tanda eksplorasi yang luas.
- Jarang ada langkah yang mengubah skor, langkah lebih baik diterima pada 1,6% iterasi (value) dan 0,5% (ETA-weighted). Langkah lebih buruk diterima pada 1,2% iterasi (value) dan 0,06% (ETA-weighted).

6.2.2 Probabilitas penerimaan e^(ΔE/T)

Langkah lebih buruk (ΔE < 0) diterima SA dengan probabilitas e^(ΔE/T). Tabel 5.3 merangkum probabilitas tersebut dari output (rata-rata per rentang iterasi, gabungan 3 seed). Plot e^(ΔE/T) terhadap iterasi untuk tiap run ada di Bab Eksperimen dan Hasil.

- Probabilitas rata-rata sekitar 0,80–0,85 pada 200 iterasi pertama, sehingga hampir semua langkah buruk diterima. Nilainya turun menjadi 0,05–0,15 setelah iterasi 500 dan mendekati nol setelah iterasi 1.000 (hanya 1 dari 968

kandidat buruk yang diterima). Langkah buruk terakhir diterima pada iterasi ke-470 sampai ke-1.078.

- Probabilitas sudah kecil sejak awal (rata-rata paling tinggi 0,058) karena selisih skor langkah buruk mencapai ribuan, jauh di atas T0 = 500. Hanya 11 dari 2.537 kandidat buruk yang diterima, semuanya sebelum iterasi ke-113. Pada objective ini SA praktis berjalan seperti stochastic hill-climbing.
- Setelah sekitar iterasi 1.000, SA praktis tidak dapat lagi melewati lembah, sehingga run yang sudah berada di local optimum tetap di sana.

6.2.3 Frekuensi stuck

Satu episode stuck dihitung ketika ada stuck_threshold = 100 iterasi berturut-turut tanpa best baru, dan berakhir ketika best baru ditemukan.Satu episode hanya dihitung sekali dan berakhir ketika best baru ditemukan. Frekuensi stuck adalah jumlah episode seperti itu dalam satu run. Penghitung bertambah pada setiap iterasi tanpa perbaikan best, termasuk saat SA menerima atau menolak langkah buruk dan saat tidak ada neighbor valid. Iterasi tanpa neighbor valid tidak memengaruhi hasil, karena neighbor_failed bernilai 0 pada seluruh run.

Ambang 100 iterasi setara 5% dari 2.000 iterasi, sehingga sebanding dengan anggaran iterasi. Ambang ini juga memisahkan stagnasi dari jeda normal. Pada 18 run, jeda antar dua perbaikan best berturut-turut bernilai median 22 iterasi, jadi 100 iterasi sekitar 4,5 kali jeda tipikal. Dari 141 jeda, 31 (22%) mencapai 100 iterasi atau lebih. Ambang 50 akan menghitung 48 jeda sebagai stuck, termasuk jeda yang masih tergolong biasa. Ambang 200 hanya menangkap 21 jeda dan akan melewatkan banyak stagnasi pada run sependek 2.000 iterasi. Ambang yang sama dipakai pada semua run, sehingga angka stuck hanya dibandingkan pada ambang yang sama.

Testcase	Objective	Frekuensi stuck	Episode yang berakhir perbaikan	Iterasi best terakhir
small	value	2 / 1 / 2	1 / 0 / 1	370 / 0 / 201
small	ETA	1 / 1 / 1	0 / 0 / 0	138 / 0 / 45
medium	value	4 / 3 / 3	3 / 2 / 2	1.311 / 473 / 598
medium	ETA	2 / 2 / 2	1 / 1 / 1	416 / 851 / 371
large	value	3 / 5 / 6	2 / 4 / 5	761 / 1.283 /

				1.850
large	ETA	4 / 3 / 4	3 /2 / 3	1.348 / 1.511 / 1.440

- Setiap run stuck minimal sekali, best terakhir paling lambat ditemukan pada iterasi ke-1.850, sehingga di akhir run selalu ada lebih dari 100 iterasi tanpa perbaikan.
- Pada seluruh 18 run, frekuensi stuck dikurangi satu sama dengan jumlah episode yang berakhir dengan perbaikan, yaitu SA berhasil keluar dari dataran tersebut.
- Large value paling sering stuck, dan best terakhirnya ditemukan sampai iterasi ke-1.850, sehingga 2.000 iterasi belum tentu cukup untuk testcase ini.

6.2.4 Dinamika Pencarian Genetic Algorithm

Pada ketiga seed dan ketiga ukuran populasi di medium, skor terbaik naik cepat di awal lalu mendatar, sedangkan rata-rata populasi terus naik tetapi tetap berada di bawah skor maksimum.

Konfigurasi	Iterasi saat skor akhir tercapai (seed 1 / 2 / 3)	Rata-rata populasi, awal ke akhir (seed 1 / 2 / 3)
populasi 20, iterasi 100	9 / 11 / 0	505 ke 575 / 499 ke 617 / 461 ke 595
populasi 50, iterasi 200	7 / 15 / 3	482 ke 590 / 495 ke 589 / 476 ke 570
populasi 100, iterasi 100	10 / 4 / 0	476 ke 628 / 488 ke 581 / 482 ke 634

Selisih antara skor maksimum dan rata-rata populasi di akhir pencarian berkisar 11 sampai 99 poin. Artinya populasi tidak menjadi seragam. Mutasi dengan peluang 0,7 dan crossover yang membuang paket bentrok terus menghasilkan individu yang lebih buruk dari parent-nya, sementara elitisme menjaga individu terbaik tetap ada. Hipotesis bahwa GA berhenti karena keragaman populasi habis (konvergensi dini) kurang didukung data ini. Penjelasan yang lebih masuk akal adalah pencarian macet di susunan terbaik yang bisa dicapai operator yang ada.

Sebagai contoh statistik operator, pada medium populasi 50 iterasi 100 seed 1, tercatat 3.445 mutasi dengan 3.058 di antaranya mengubah state (88,8%), dan 3.946 crossover dengan 3.042 di antaranya menghasilkan child yang berbeda dari kedua parent (77,1%). Sekitar 23% crossover menghasilkan child yang sama dengan salah satu parent, kemungkinan karena kedua parent mirip atau hasil penggabungannya akhirnya identik dengan salah satu parent.

6.2.5 Dinamika Pencarian Steepest Ascent Hill Climbing

Testcase	Objective	Perpindahan Meningkat
Small	Value	1 / 0 / 2
Small	ETA	1 / 0 / 4
Medium	Value	0 / 1 / 2
Medium	ETA	1 / 2 / 3
Large	Value	6 / 3 / 4
Large	ETA	5 / 4 / 5

Hasil:

- Pencarian hanya menerima 0-6 perpindahan per run. Seluruh run berhenti sebelum batas 1000 iterasi karena tidak ditemukan neighbor dengan skor lebih tinggi.
- State awal menentukan peluang perbaikan: Medium Value seed 42 langsung berhenti pada 488, sedangkan Small Value seed 43 langsung berhenti pada 344. Nol perpindahan bukan berarti kualitas solusi selalu rendah.
- Pada Large Value seed 42, skor berkembang 846 → 906 → 962 → 982 → 1001 → 1017 → 1025. Peningkatan semakin kecil pada run ini hingga pencarian tidak dapat melanjutkan tanpa langkah netral atau penurunan skor.

6.2.6 Dinamika Pencarian Sideways Move Hill Climbing

Testcase	Objective	Total Iterasi	Langkah meningkat	Langkah setara	Peningkatan terakhir
Small	Value	133 / 100 / 108	3 / 0 / 3	130 / 100 / 105	33 / 0 / 8
Small	ETA	133 / 100 / 104	3 / 0 / 4	130 / 100 / 100	33 / 0 / 4
Medium	Value	114 / 149 / 109	6 / 4 / 3	108 / 145 / 106	14 / 49 / 9
Medium	ETA	174 / 125 / 216	6 / 3 / 5	168 / 122 / 211	74 / 25 / 116

Large	Value	110 / 205 / 186	7 / 8 / 7	103 / 197 / 179	10 / 105 / 86
Large	ETA	105 / 180 / 237	5 / 6 / 11	100 / 174 / 226	5 / 80 / 137

Hasil:

- Langkah setara mendominasi pencarian. Susunan paket dapat berubah tanpa mengubah objective, tetapi perubahan tersebut membuka peluang perbaikan berikutnya.
- Small Value seed 42 mencapai 228 pada iterasi 1, menjalani 30 langkah setara, lalu mencapai 327 pada iterasi 32 dan 344 pada iterasi 33. Steepest Ascent dari state awal yang sama berhenti pada 228.
- Seluruh run menghabiskan 100 langkah terakhir tanpa peningkatan. Batas 100 berlaku berturut-turut dan direset setelah perbaikan, sehingga total sideways dapat melebihi 100.
- Small seed 43 menjalankan 100 langkah setara tanpa peningkatan pada kedua objective. Eksplorasi plateau tidak selalu menghasilkan solusi lebih baik.

6.2.7 Dinamika Pencarian Random Restart Hill Climbing

Testcase	Objective	Total perpindahan lokal	Indeks akhir riwayat
Small	Value	11 / 14 / 15	21 / 24 / 25
Small	ETA	17 / 14 / 19	27 / 24 / 29
Medium	Value	18 / 23 / 27	28 / 33 / 37
Medium	ETA	30 / 33 / 36	40 / 43 / 46
Large	Value	63 / 63 / 58	73 / 73 / 68
Large	ETA	62 / 67 / 54	72 / 77 / 64

Hasil:

- Restart menjelajahi daerah solusi lain setelah pencarian lokal berhenti. Medium Value seed 42 berhenti pada 488 tanpa perpindahan di pencarian awal, tetapi keseluruhan run akhirnya menemukan 680.
- Grafik menyimpan skor terbaik sejauh ini, sehingga tidak menunjukkan penurunan ketika restart dimulai. Grafik mendatar dapat mencakup pencarian lokal yang membaik tetapi belum melampaui solusi terbaik sebelumnya.
- Small Value seed 43 memiliki grafik tetap 344 meskipun melakukan 14 perpindahan lokal selama restart. Grafik mendatar tidak berarti pencarian tidak berlangsung.

- Nilai iterations mencakup perpindahan lokal dan 10 transisi restart. Gunakan jumlah iterations_per_restart untuk total perpindahan lokal; jangan menafsirkan semua indeks riwayat sebagai perpindahan lokal.

6.2.8 Dinamika Pencarian Stochastic Hill Climbing

Testcase	Objective	Perpindahan diterima	Peningkatan terakhir	% diterima
Small	Value	1 / 0 / 3	22 / 0 / 24	0,13%
Small	ETA	1 / 0 / 8	22 / 0 / 55	0,30%
Medium	Value	0 / 1 / 3	0 / 42 / 137	0,13%
Medium	ETA	1 / 2 / 4	84 / 47 / 76	0,23%
Large	Value	8 / 4 / 13	121 / 117 / 638	0,83%
Large	ETA	8 / 9 / 5	258 / 317 / 240	0,73%

Hasil:

- Sebagian besar iterasi tidak mengubah state. Persentase perpindahan diterima kurang dari 1% pada semua kelompok pengujian karena hanya kandidat dengan skor lebih tinggi yang diterima.
- Large Value seed 44 mengalami peningkatan terakhir pada iterasi 638, menyisakan 362 iterasi tanpa perbaikan. Small Value seed 42 sudah mencapai skor akhirnya pada iterasi 22, menyisakan 978 iterasi tanpa peningkatan.
- Medium Value seed 44 mencapai 616, sedikit lebih tinggi daripada Steepest Ascent yang mencapai 613. Jalur peningkatan acak dapat menghasilkan solusi akhir berbeda dari pemilihan peningkatan terbesar.
- Berhenti pada batas iterasi belum membuktikan optimum lokal karena sampling belum tentu mencoba seluruh neighbor. Iterasi tanpa perbaikan juga tidak membuktikan bahwa tidak ada neighbor yang lebih baik.

6.3 Pengaruh Iterasi dan Jumlah Populasi pada GA

Pada eksperimen A (populasi 50), skor akhir untuk iterasi 50, 100, dan 200 sama persis pada setiap seed (664, 680, 645). Hal ini terjadi karena skor tersebut sudah tercapai pada iterasi 3 sampai 15, dan karena seed yang sama menghasilkan lintasan yang sama, iterasi tambahan hanya memperpanjang lintasan yang sudah mendatar. Durasi naik dari 1,31 s ke 1,95 s dan 3,25 s, kira-kira linear terhadap jumlah iterasi. Pada konfigurasi ini, menambah iterasi hanya menambah waktu tanpa menambah kualitas.

Pada eksperimen B (iterasi 100), jumlah populasi berpengaruh lebih besar. Rata-rata skor akhir naik dari 624,0 (populasi 20) ke 663,0 (populasi 50) dan 680,0 (populasi 100), dan populasi 100 mencapai 680 pada ketiga seed. Durasinya naik dari 0,78 s ke 1,95 s dan 3,71 s. Namun sebagian besar kenaikan ini berasal dari titik awal yang lebih baik. Rata-rata skor awal (terbaik di populasi awal) adalah 603,3, 636,7, dan 656,3 untuk populasi 20, 50, dan 100, sedangkan rata-rata kenaikan yang dihasilkan proses pencarian relatif mirip, yaitu 20,7, 26,3, dan 23,7 poin (dihitung dari tabel 5.3.4). Jadi populasi yang lebih besar terutama membantu karena mengambil lebih banyak state awal acak, bukan karena pencarian di dalamnya menjadi jauh lebih efektif. Pada populasi 100 seed 3, skor awalnya sudah 680 sehingga tidak ada peningkatan sama sekali.

6.4 Perbandingan Antar Algoritma

Hasil. Rata-rata skor akhir seluruh algoritma (tiga seed) pada testcase resmi ditunjukkan pada tabel berikut. Perbandingan skor antar algoritma dibatasi pada objective Value karena eta_bonus pada HC (100) berbeda dengan SA dan GA (300), sehingga skor ETA HC tidak sebanding secara langsung.

Testcase	Objective	Steep-HC	Sideways	RR-HC	Stokastik	SA	GA
small	value	279,3	327,0	344,0	279,3	344,0	344,0
small	ETA	14.632,0	17.034,0	17.034,0	14.632,0	50.414,0	50.414,0
medium	value	548,7	640,3	654,7	549,7	613,7	663,0
medium	ETA	16.541,6	19.023,3	20.934,0	16.451,6	58.184,0	62.224,0
large	value	1.008,7	1.042,3	1.038,3	1.013,0	981,0	1.041,7
large	ETA	22.777,9	23.407,3	23.587,5	22.796,3	67.068,9	69.832,7

Pada small, Random Restart, SA, dan GA sama-sama mencapai 344,0, sedangkan Sideways Move 327,0 serta Steepest Ascent dan Stochastic 279,3. Pada medium, GA tertinggi (663,0), diikuti Random Restart (654,7), Sideways Move (640,3), dan SA (613,7), sedangkan Steepest Ascent (548,7) dan Stochastic (549,7) paling rendah. Pada large, Sideways Move (1.042,3) dan GA (1.041,7) hampir sama, diikuti Random Restart (1.038,3), Stochastic (1.013,0), Steepest Ascent (1.008,7), dan SA (981,0). Jadi GA tidak selalu terbaik. Pada large selisihnya dengan Sideways Move hanya 0,6

poin (sama-sama 96,3% dari optimum). Selisih GA dengan SA sekitar 49 poin pada medium value dan 61 poin pada large value, dan GA juga lebih tinggi daripada SA pada kedua objective di medium dan large.

Hasil ini perlu dibaca dengan beberapa catatan. Seed HC (42, 43, 44) berbeda dengan seed SA dan GA (1, 2, 3), sehingga state awal HC berbeda dan perbandingan dilakukan pada rata-rata dan rentang tiga run, bukan per seed. Jumlah state yang dievaluasi juga tidak sama. GA mengevaluasi sekitar 5.050 state, SA sekitar 2.000 state, sedangkan HC mengevaluasi seluruh neighbor pada setiap iterasinya.

Durasi. SA paling cepat (0,34 sampai 0,43 s pada small dan 0,76 sampai 1,67 s pada large), disusul Stochastic (0,1 sampai 0,35 s pada semua testcase). GA berdurasi sekitar 1,0 s (small), 1,8 sampai 2,1 s (medium), dan 8,2 sampai 9,6 s (large). Steepest Ascent berdurasi 0,015 sampai 0,26 s pada small dan medium, tetapi 1,9 sampai 3,0 s pada large. Random Restart berdurasi 0,29 sampai 0,50 s (small), 1,1 sampai 2,1 s (medium), dan 20 sampai 34 s (large). Sideways Move paling lambat, yaitu 1,4 sampai 3,3 s (small), 5,8 sampai 10,1 s (medium), dan 46 sampai 95 s (large), karena setiap iterasinya mengevaluasi seluruh neighbor dan masih berlanjut pada langkah setara. GA lebih lambat daripada SA karena mengevaluasi populasi penuh tiap generasi dan crossover memanggil is_valid berulang kali.

Konsistensi. Pada objective Value, rentang skor antar seed GA adalah 0, 35, dan 18 pada small, medium, dan large. Steepest Ascent dan Stochastic paling bervariasi, dengan rentang 116 sampai 128 poin pada small dan medium. Random Restart konsisten pada small (344 pada semua seed). Pada large, GA memiliki rentang terkecil (18), diikuti Sideways Move (21), Random Restart (28), SA (26), dan Steepest Ascent serta Stochastic (39 dan 42). SA paling sempit pada medium (7), tetapi pada level skor yang lebih rendah. Dengan hanya tiga seed, hasil ini lebih tepat dibaca sebagai kecenderungan.

6.5 Pengaruh Objective ETA-weighted dan Multi-Truk

Testcase	Value murni (value ke ETA)	Rata-rata ETA terangkut (value ke ETA)	Paket termuat (value ke ETA)
small	344 ke 344	1,60 ke 1,60	5,00 ke 5,00
medium (populasi 50, iterasi 100)	663,0 ke 632,3	4,45 ke 3,82	7,00 ke 7,33

large	1.041,7 ke 879,3	4,91 ke 3,83	15,67 ke 14,33

Mode eta_weighted menggeser pemilihan ke paket ber-ETA kecil, dengan konsekuensi value murni turun sekitar 4,6% pada medium dan 15,6% pada large. Ini trade-off yang wajar karena objective-nya memang tidak lagi hanya memaksimalkan value. Pada small tidak ada perubahan karena paket bernilai tinggi kebetulan juga ber-ETA kecil. Pada kasus multi-truk, semua truk selalu terpakai pada seluruh run GA, dan durasi naik dari sekitar 1 s pada small (1 truk, 10 paket) ke sekitar 9 s pada large (3 truk, 30 paket).

6.6 Keterbatasan

Eksperimen resmi hanya memakai tiga seed per konfigurasi sehingga hasilnya merupakan kecenderungan, bukan bukti statistik. Eksperimen parameter hanya dilakukan pada satu testcase (medium) dan satu objective (value), dan hanya populasi serta iterasi yang divariasikan, sementara tournament_size, mutation_rate, dan crossover_rate tetap. Nilai optimum berasal dari skrip tersendiri, sehingga validitasnya bergantung pada kesamaan aturan constraint dengan program utama. Perbandingan durasi dengan SA dan HC dipengaruhi perbedaan parameter dan perbedaan jalur eksekusi. Skor mode value dan eta_weighted tidak setara sehingga tidak dibandingkan langsung. Eksperimen HC memakai seed 42 sampai 44 dan eta_bonus 100, sedangkan SA dan GA memakai seed 1 sampai 3 dan eta_bonus 300, sehingga skor ETA HC tidak dibandingkan dengan SA dan GA. Jumlah state yang dievaluasi tiap algoritma juga tidak setara. Testcase yang dipakai juga bersifat sintetis.

BAB VII
KESIMPULAN DAN SARAN

7.1 Kesimpulan

Pada pengujian Hill Climbing, Sideways Move dan Random Restart menghasilkan rata-rata objective VALUE lebih tinggi daripada Steepest Ascent dan Stochastic pada seluruh ukuran testcase. Random Restart memberikan rata-rata VALUE tertinggi di antara varian HC pada small (344) dan medium (654,67), sedangkan Sideways Move unggul pada large (1.042,33). Hasil ini menunjukkan manfaat eksplorasi plateau dan penggunaan beberapa state awal, meskipun keduanya memerlukan durasi lebih besar. Random Restart mencapai VALUE 344 pada seluruh seed small, sementara Sideways Move masih menghasilkan 293 pada salah satu seed. Steepest Ascent berhenti setelah 0–6 perpindahan pada pengujian ini, tetapi kualitas hasilnya bergantung pada state awal. Stochastic menjalankan 1.000 iterasi, namun tambahan iterasi tersebut belum menghasilkan perbaikan yang konsisten dibandingkan Steepest Ascent. Pada objective ETA Weighted, Random Restart unggul pada medium (20.934,0) dan large (23.587,5), sedangkan pada small Sideways Move dan Random Restart sama (17.034,0). Dengan demikian, tidak ada satu varian yang selalu memberikan hasil terbaik untuk seluruh test case dan objective.

Simulated Annealing mencapai optimum eksak pada semua run testcase small (6 dari 6 run, termasuk satu seed yang state awalnya sudah optimal). Pada medium dan large tidak ada run yang mencapai optimum, dengan skor rata-rata 90,2–94,0% dari optimum eksak. Seluruh state akhir terbukti local optimum, karena tidak ada neighbor satu langkah yang lebih baik. Keluar dari sana membutuhkan beberapa langkah turun berurutan, misalnya mengeluarkan beberapa paket sekaligus agar paket bernilai tinggi muat. Pada objective value, langkah turun hampir selalu diterima di awal pencarian dan hampir tidak pernah setelah iterasi 1.000. Pada objective ETA-weighted, T0 = 500 jauh di bawah skala ΔE (ribuan), sehingga hanya 11 dari 2.537 langkah buruk yang diterima dan SA berjalan seperti stochastic hill-climbing. Sekitar 85–89% langkah bersifat netral (ΔE = 0), sehingga acceptance rate yang tinggi tidak menunjukkan eksplorasi yang luas. Hasil akhir konsisten pada small, tetapi bervariasi pada medium ETA-weighted (79,7–95,6% optimum) dan large. Frekuensi stuck 1–6 per run, paling sering pada large value. Durasi 0,3–1,7 detik per run, dan large memerlukan waktu terlama.

Genetic Algorithm menghasilkan state valid dengan rata-rata skor 96,3 sampai 100% dari nilai optimum pada ketiga testcase. Pada objective Value, GA tertinggi di antara seluruh algoritma pada medium (663,0), hampir sama dengan Sideways Move pada large (1.041,7 dibandingkan 1.042,3), dan lebih tinggi daripada SA pada medium dan large, dengan konsekuensi durasi dan jumlah evaluasi yang lebih besar daripada SA. Pada konfigurasi yang diuji, jumlah populasi berpengaruh lebih besar daripada jumlah iterasi. Iterasi di atas 50 tidak menambah kualitas pada testcase medium karena skor terbaik sudah tercapai pada iterasi awal, sedangkan populasi yang lebih besar menaikkan skor akhir, sebagian besar karena titik awal pencarian yang lebih baik. Fitur bonus berupa objective ETA-weighted dan multi-truk berfungsi pada GA. Seluruh truk yang tersedia terpakai pada semua run, dan objective ETA-weighted menggeser pemilihan ke paket ber-ETA kecil dengan konsekuensi value murni yang lebih rendah.

7.2 Saran

Pengujian selanjutnya sebaiknya memvariasikan maximum sideways move dan maximum restart untuk menilai keseimbangan antara kualitas solusi dan durasi. Neighborhood juga dapat diperluas dengan langkah gabungan, seperti mengeluarkan beberapa paket sebelum memasukkan paket bernilai lebih tinggi, agar pencarian dapat mencapai susunan yang sulit diperoleh melalui satu perpindahan. Jumlah seed perlu ditambah agar penilaian konsistensi lebih kuat. Perbandingan efisiensi sebaiknya menggunakan batas waktu atau jumlah evaluasi objective yang setara karena biaya satu iterasi berbeda antarvarian. Untuk Stochastic, penghentian setelah sejumlah iterasi tanpa perbaikan dapat diuji guna mengurangi pencarian yang tidak produktif. Rumus dan parameter ETA Weighted juga perlu diseragamkan sebelum skor HC dibandingkan langsung dengan SA dan GA.

Pertama, karena hanya satu konfigurasi yang dijalankan, pengaruh parameter perlu diuji: T0 mendekati median $|\Delta E|$ (puluhan sampai ratusan untuk value, ribuan untuk ETA-weighted, atau ΔE dinormalisasi terhadap skala skor), serta variasi α. Kedua, large masih menemukan best baru sampai iterasi ke-1.850, jadi jumlah iterasi perlu ditambah. Ketiga, neighbor dapat diperluas dengan langkah gabungan, misalnya mengeluarkan beberapa paket kecil lalu memasukkan satu paket bernilai tinggi, agar lembah yang membutuhkan beberapa langkah turun lebih mudah dilewati. Keempat, jalankan lebih banyak seed atau restart untuk menurunkan variasi hasil, dan bandingkan dengan Hill-climbing dan GA pada testcase yang sama.

Eksperimen sebaiknya diperluas dengan lebih banyak seed dan lebih dari satu testcase, terutama untuk eksperimen parameter. Parameter tournament_size, mutation_rate, dan crossover_rate juga dapat divariasikan secara resmi untuk melihat pengaruhnya. Dari sisi algoritma, dapat dicoba teknik yang menjaga keragaman populasi serta operator crossover yang kehilangan lebih sedikit value, karena child hasil crossover cenderung lebih buruk daripada parent-nya akibat paket yang bentrok dibuang.

PEMBAGIAN TUGAS

NIM	Tugas
13524008	- Implementasi Hill Climbing Algorithm - Implementasi bonus objective function - Implementasi bonus seluruh algoritma Hill Climbing - Penulisan laporan
13524098	- Implementasi Simulated Annealing Algorithm - Implementasi frontend - Penulisan laporan
13524131	- Implementasi Genetic Algorithm - Implementasi bonus multi-truck - Penulisan laporan

REFERENSI

[1] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Okt. 19, 2026].

[2] Salindia kuliah IF3170 Inteligensi Artifisial, “Beyond Classical Search,” Institut Teknologi Bandung, Semester I TA 2026/2027. [Slide kuliah]. (Memuat ilustrasi dan pseudocode dari Russell & Norvig, 2010.)

[3] [3] S. Kirkpatrick, C. D. Gelatt Jr., dan M. P. Vecchi, “Optimization by Simulated Annealing,” Science, vol. 220, no. 4598, pp. 671–680, 1983. [Online]. Available: https://doi.org/10.1126/science.220.4598.671 [Accessed: Okt. 19, 2026].

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: Agatha Tatianingseto/13524008 Reva Natania Sitohang/13524098 Amanda Aurellia Salsabilla/13524131
Fakultas/Sekolah	: STEI-K
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Intelegensi Artifiisal
Judul Tugas/Proposal	: Tugas Besar 1

Menyatakan bahwa kami menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Rendah
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Rendah
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026







Agatha Tatianingseto 13524008 Reva Natania Sitohang 13524098 Amanda Aurellia Salsabilla 13524131